

Workshop: Build Your Own YOLO Tracker

Objectives

After finishing this workshop, you should:

1. Be able to build a human tracking robot on your own
2. Learn the basics of how computer vision works
3. Enjoy building Robots !!!

Estimated Time

- **1.5 to 2 hours**

YOLO tracker and workshop notes designed by Mr. Kelvin Chan

Ideas based on ENGG1101 mini-project (YOLO tracker on drones) by Dr. Ryan Wang

Present by: HKU Active Learning Team

(Lectures: Dr. Ryan Wang, Dr. Edwin Dung, Dr. Timmy Cheng, Mr. Kyle Ho)

(TAs: Mr. Kelvin Chan, Mr. Karlos Ma, Mr. Sum Cheung, Mr. Henry Tang)

Warm Up & Form Your Team

Find your Teammates and collect the materials

1. Collect your workshop materials from the teaching team

2. Introduce yourselves to your teammates

You should get a material box with the following items:



Material Box Bill of Materials (MB-BOM):

<u>Items</u>	<u>Amount</u>	<u>Remarks</u>
3D Printed parts	9	Check if you have all the different types of 3D printed part as shown
Servo Motor Wires	2	260mm long 3 pin wire
Bearing	1	30*42*7 Bearing
Hex Keys	3	M3 hex key with handles
Spanners	3	5.5mm & 7mm spanner
USB-C Cable	1	

Check if you got all the materials, if not please raise your hand and ask the teaching team.

Other than the material box, you should also get a set of extra materials, including:



Extra Materials Bill of Materials (EM-BOM):

<u>Items</u>	<u>Amount</u>	<u>Remarks</u>
3D Printed parts	2	Electronic Box base and cover
Servo Motors	2	12V STS3215 Servo Motors
ESP32 Servo Motor Driver	1	Black Driver Board for controlling STS3215 Servo Motors
Raspberry Pi	1	Main controlling unit
USB camera	1	
M2 Hex key	1	
Crossed Screw Driver	1	
Screws and nuts set	1	

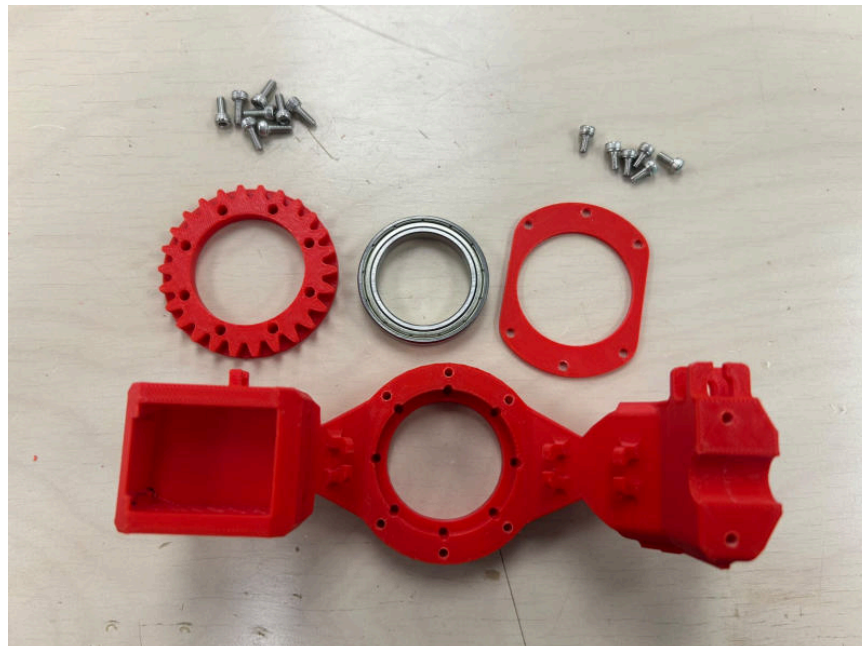
If you have checked that you have all the materials and introduce yourself to your teammates, it is time to start building your own tracker robot. Let's goooooo!

Assembling Your Human Tracker

▼ Part 1: Assembling the Rotation Part

In this part, we would start by building the middle yaw rotating part of the tracker.

You would need the following materials in this part:



<u>Items</u>	<u>Amount</u>
3D Printed U shaped mount (largest)	1

3D printed Gear part (one with flat surface)	1
3D printed cover part (cut circle)	1
Bearing	1
M3 screws (8mm long)	8
M3 Screws (6mm long)	6

First, we put the bearing in the “3D printed U shaped mount” as shown below.

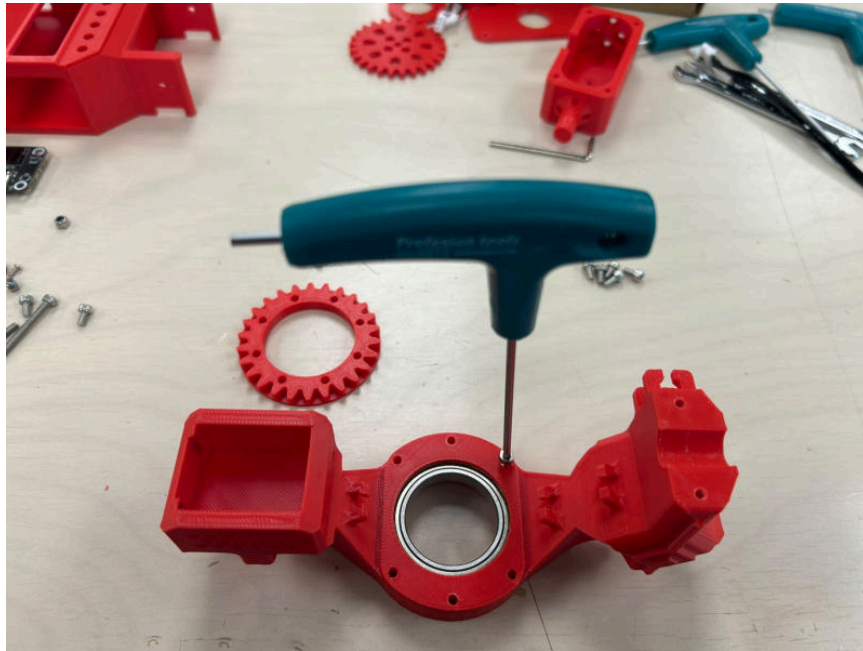
You may have to use some force to push it in.
But be careful, don't break the 3D printed part (or yourself).



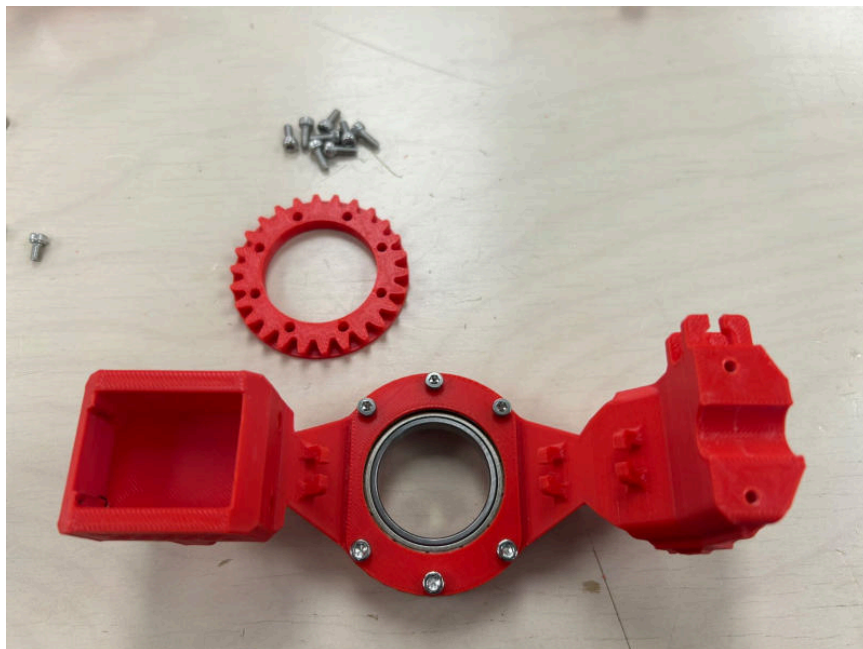
Place the “3D printed cover part” on top of the bearing, Insert 6mm long M3 screws to the 6 holes and align it with the “3D printed U shaped mount”

Screw in the part with a hex key. Tighten the parts until you feel a sudden increase of resistant force.

Screw it clockwise to tighten the parts and anti-clockwise to untighten it.



After tightening, your part should look like this:

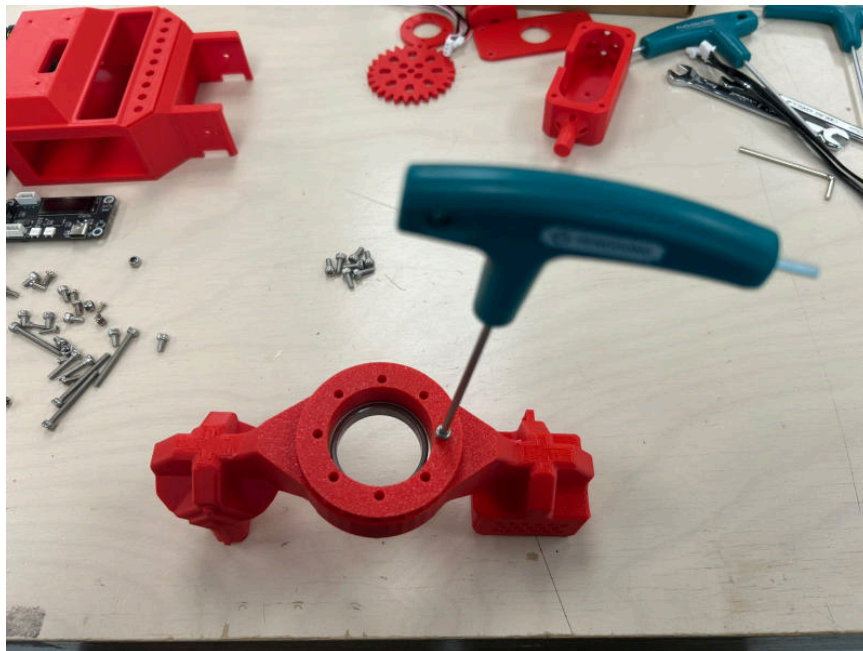


Flip the "3D printed U shaped mount" over and place the "3D printed Gear part" on top.

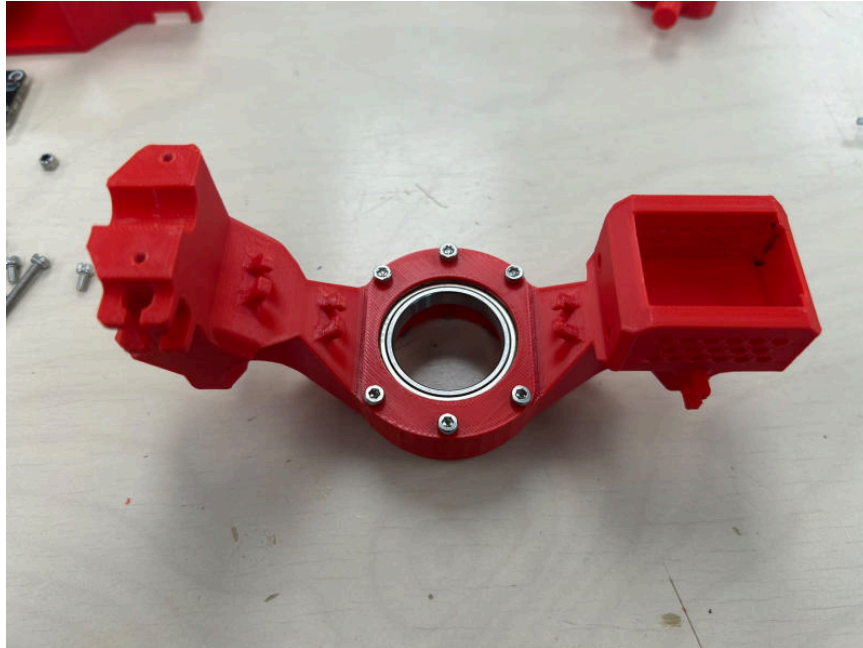
Note that there is a flat surface at the bottom of the "3D printed Gear part"

It stays at the bottom of the part, not sticking to the "3D printed U shaped mount"

Align the holes and tighten the parts using 8mm long M3 screws just like what you did.



After tightening, your part should look like this from the top with a gear attached at the bottom

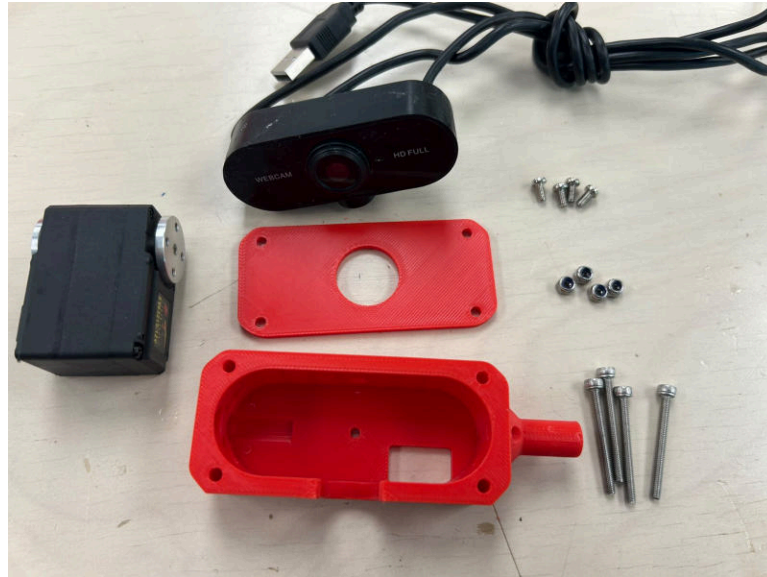


Yea, and you just finished Part 1. Good job, let's move to the next part

▼ Part 2: Assembling the Camera Mounting Part

In this part, we would building the camera mounting part and pitch rotation part of the tracker.

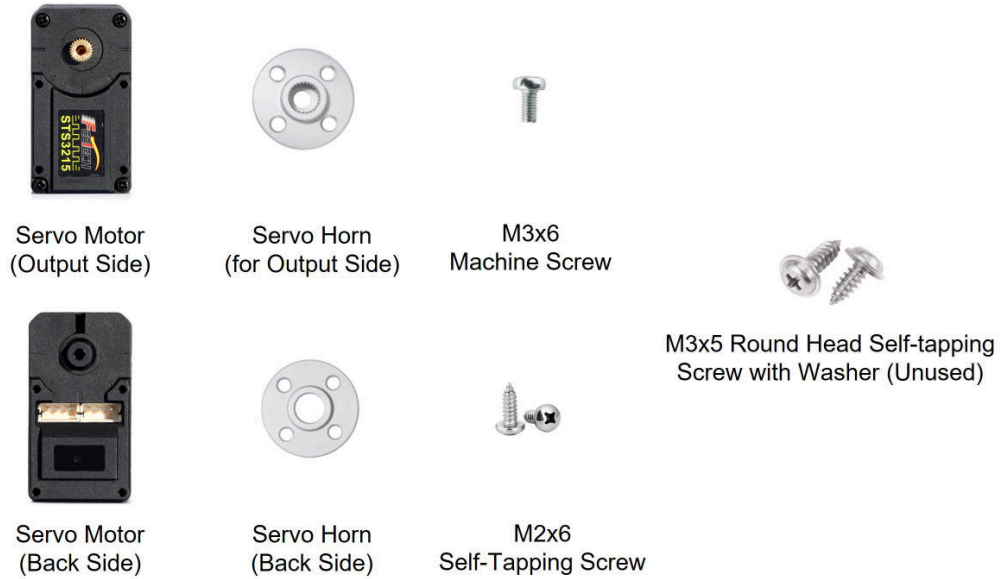
You would need the following materials in this part:



<u>Items</u>	<u>Amount</u>
3D Printed camera housing with rod	1
3D printed camera cover	1
STS3215 Servo Motor (Motor 2)	1
USB camera	1
M3 screws (30mm long)	4
Crossed Servo Motor Screws (M3*6)	4
M3 Nuts	4

Before assembling the part, notice that there are two sides of the servo motor:

- **Output Side:** the side with **Metal Gear Teeth**
- **Back Side:** the opposite side with smooth plastic



- For the output side, we need to attach the servo horn with gears (teeth).
- For the backside, we could just use a smooth servo horn acting as a bearing



Alright, back to assembling the part, we only need to install the metal servo horns on the output side for this project.

First, align the four holes on "3D Printed camera housing with rod" to the holes on the servo horn attached to the output side of the servo motor.

In this part we use MOTOR 2, which has a sticker labeled with 2 below the motor



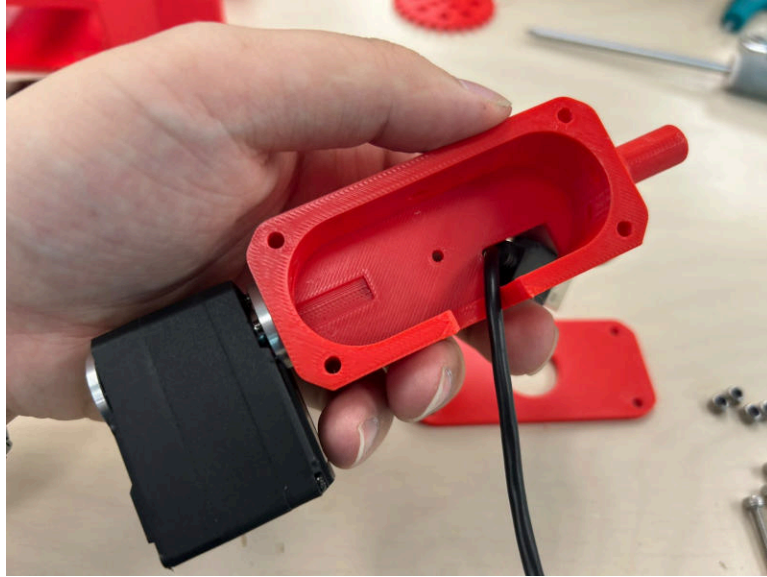
Insert the servo motor screws (cross-headed) to each of the holes and tighten them using the cross screw driver provided.



After tightening, your part should look like this:



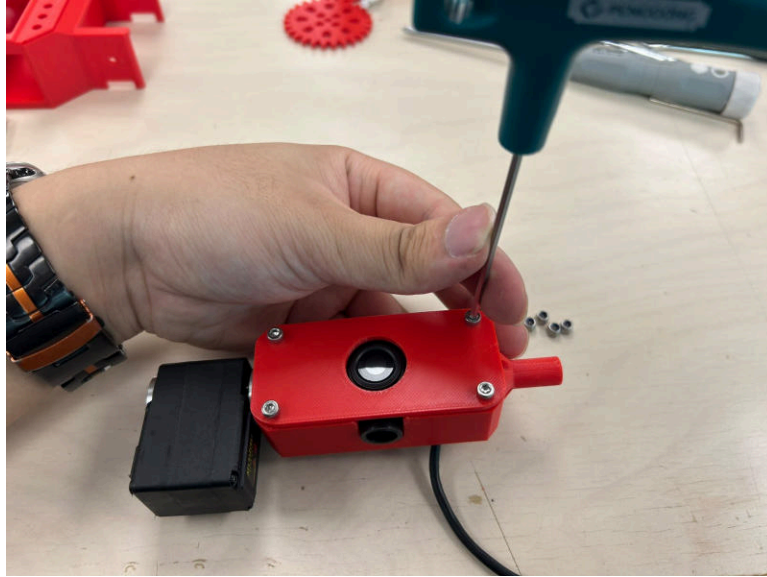
Wire the USB end of the USB camera through the square slot on the bottom right of the "3D Printed camera housing with rod"



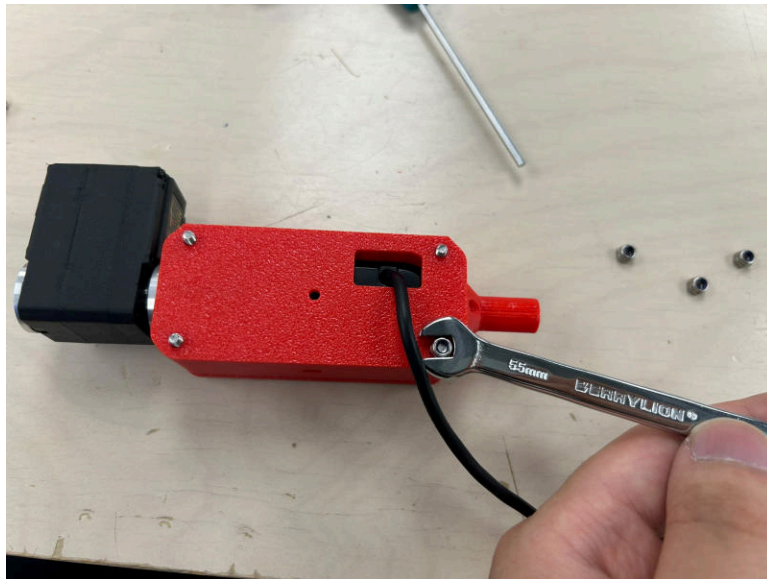
Insert the camera into the 3D printed part, it should be a tight fit.



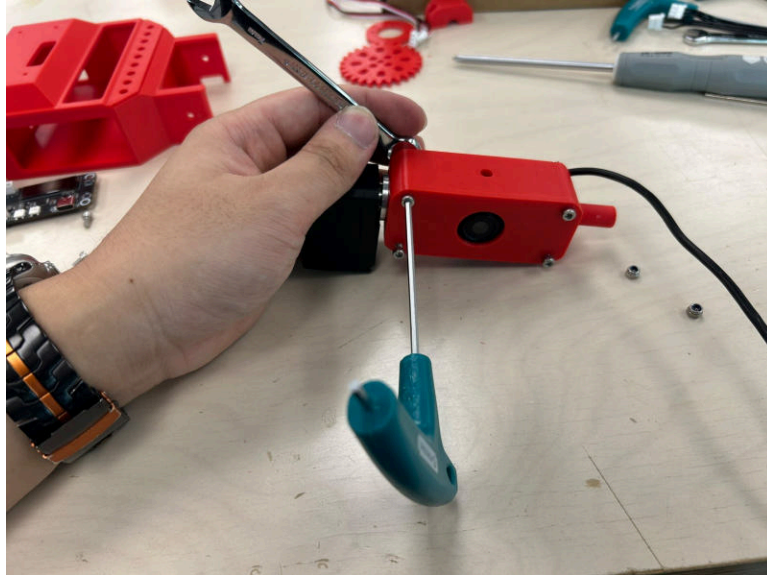
Place the "3D printed camera cover" on top and insert 4 M3 screws (30mm long) to all the holes, passing through both 3D printed parts.



Place the M3 Nuts on the screws by rotating them in clockwise (with hand or spanner)



Fix the orientation of the M3 nuts using the spanner (5.5mm end) at the back and at the same time rotate the M3 hex key clockwise to tighten the two 3D printed parts as shown:



After tightening, your part should look like this from the top with a camera protected with a 3D printed housing and a servo motor at one side.

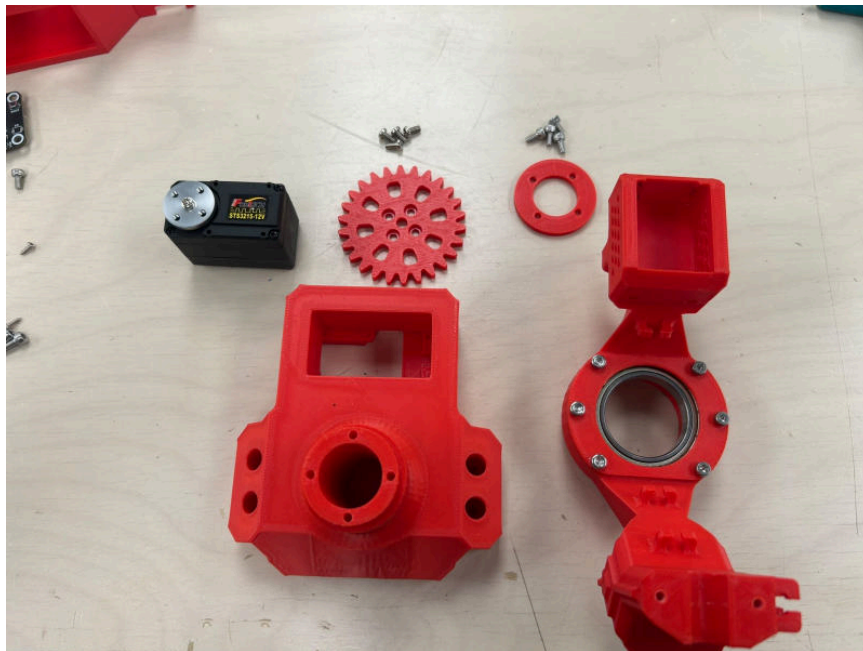


That's all for Part 2, are you enjoying it?
There's more.

▼ Part 3: Assembling the Base

In this part, we would connect the rotation part we build in Part 1 with the base

You would need the following materials in this part:

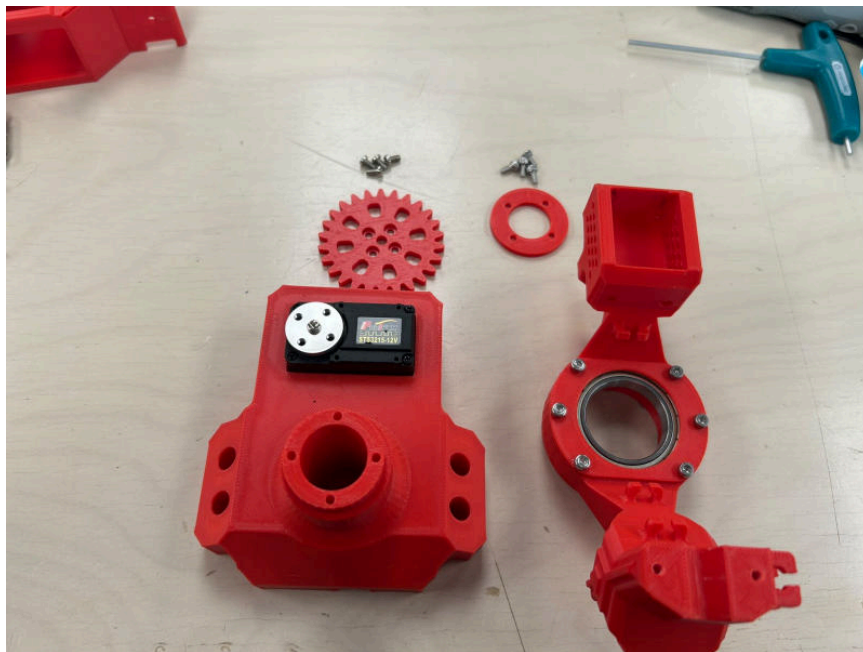


<u>Items</u>	<u>Amount</u>
Finished part from "Part 1"	1
3D printed base	1
3D printed Gear part (one without flat surface)	1
3D printed bearing cover (circle)	1
STS3215 Servo Motor (Motor 1)	1
M3 screws (6mm long)	4
Crossed Servo Motor Screws (M3*6)	4

Place the servo motor at back of the "3D printed base"

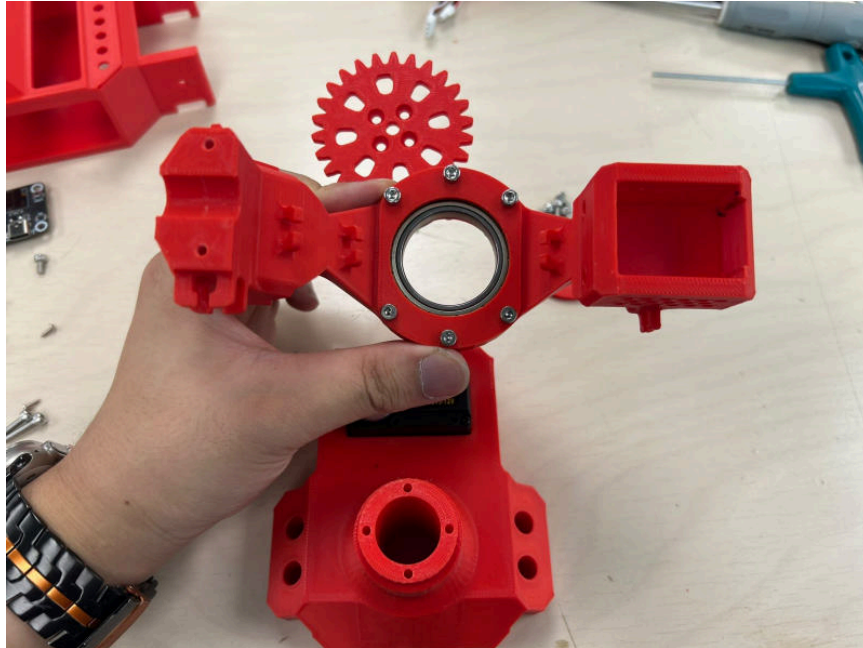
Notice the orientation of the servo motor, the metal horn should be on the left

In this part we use MOTOR 1, which has a sticker labeled with 1 below the motor

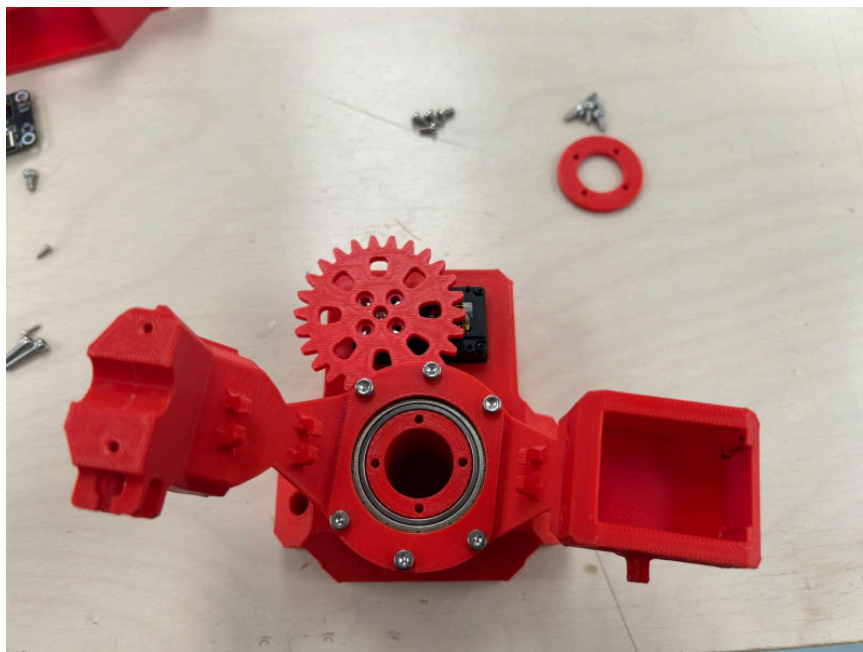


Insert the "3D printed Gear part (one without flat surface)" to the Finished part from "Part 1"

The teeth of the gears should be aligned

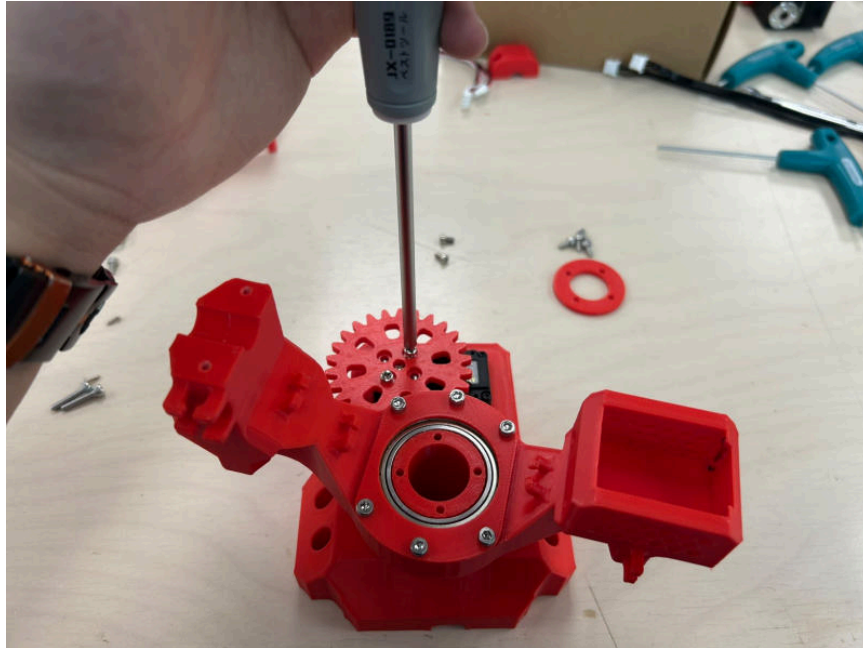


Place the whole part onto the base. The bearing should be sitting on the circle part attached to the base as shown:

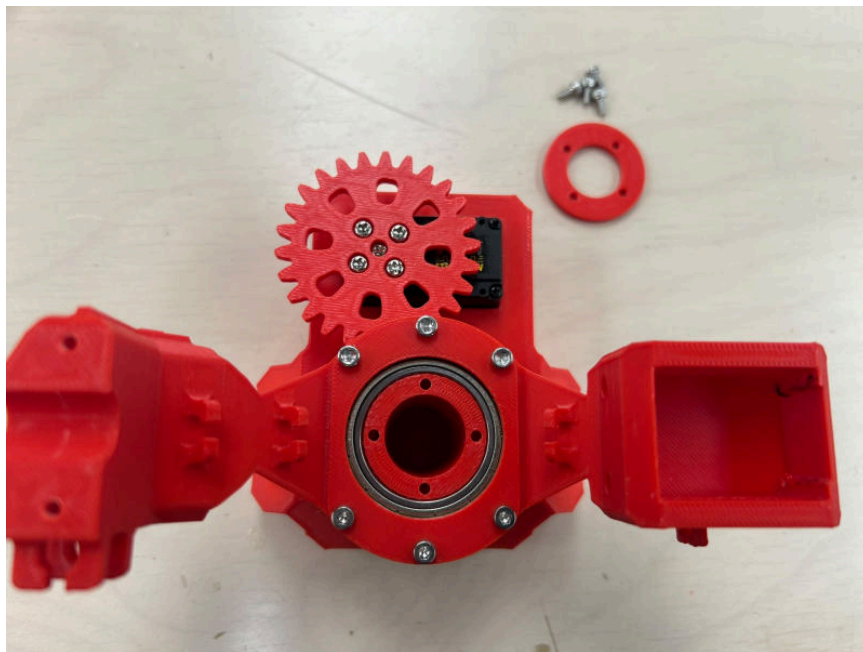


Align the holes on the 3D printed gear and the servo motor horn.

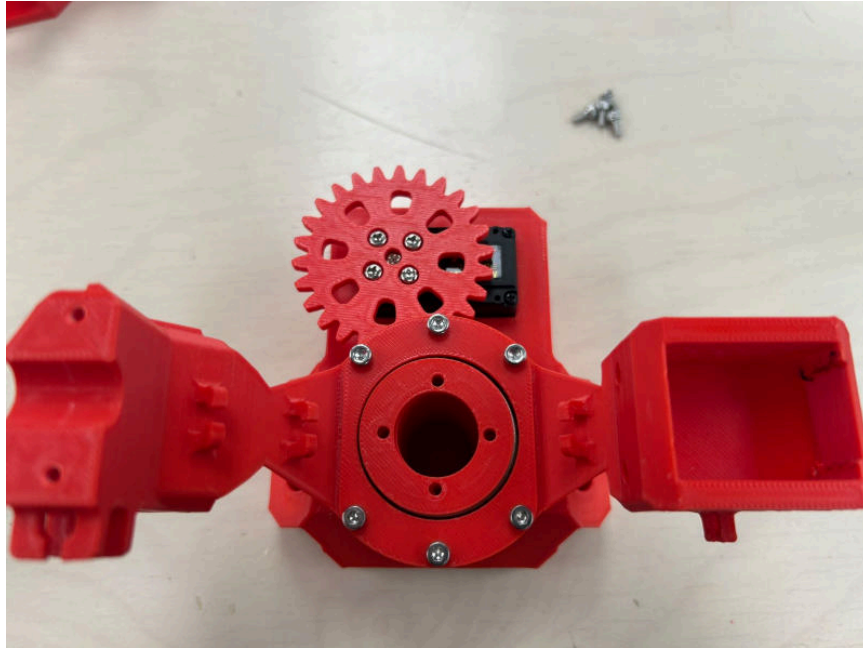
Screw in the parts with cross-headed servo motor screws using the screw driver



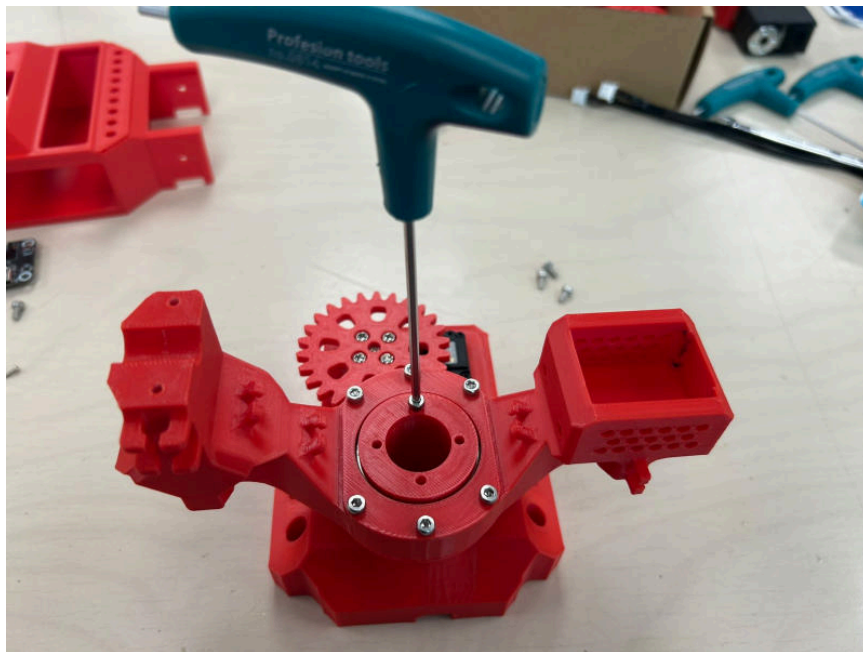
After tightening the screws, you should get a part that looks like this:



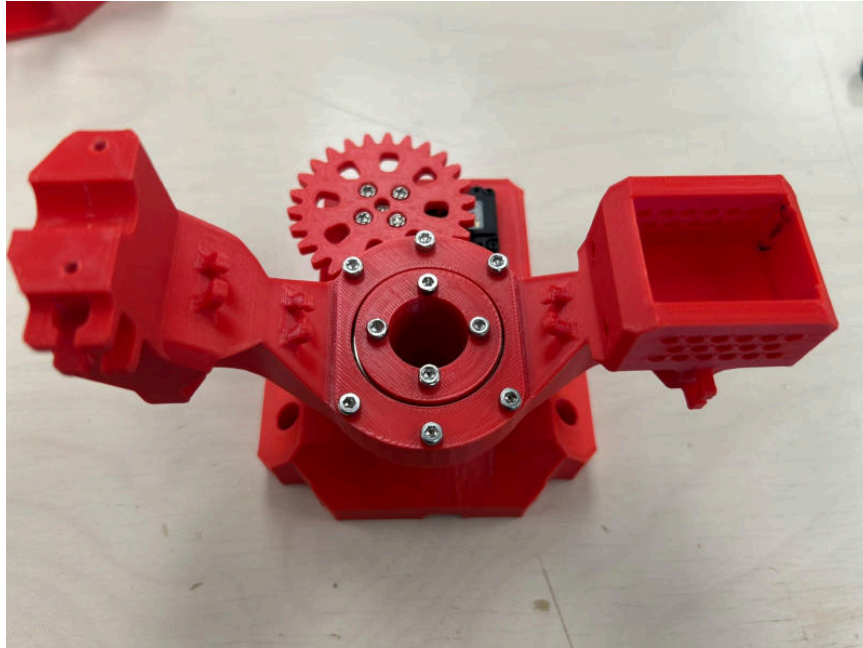
Place the "3D printed bearing cover (circle)" on top of the bearing and align the holes.



Tighten the parts with 6mm M3 screws using the hex key



After tightening the screws, you should get a part that looks like this:

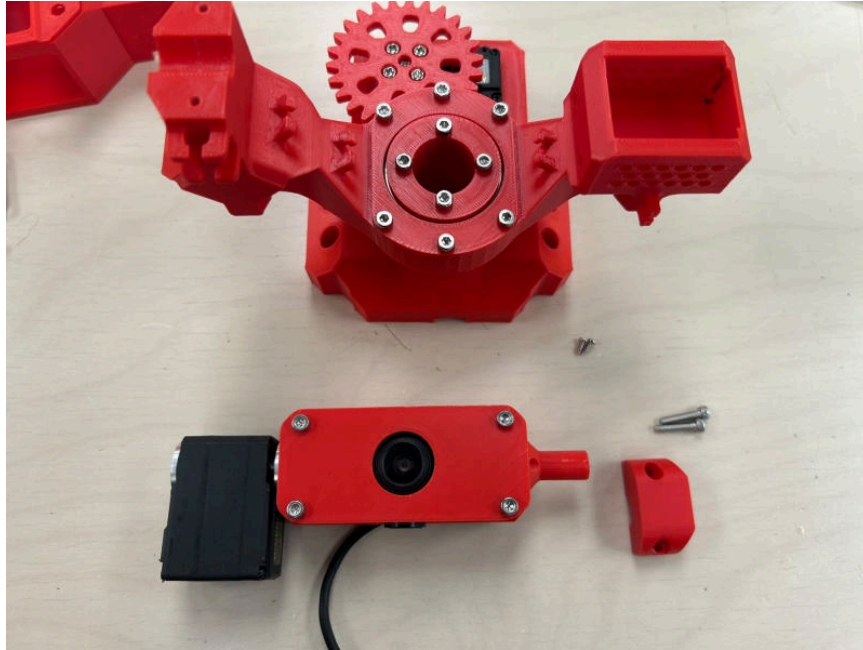


Now, you've got rotating base for the tracker.

▼ Part 4: Assembling the Main Body

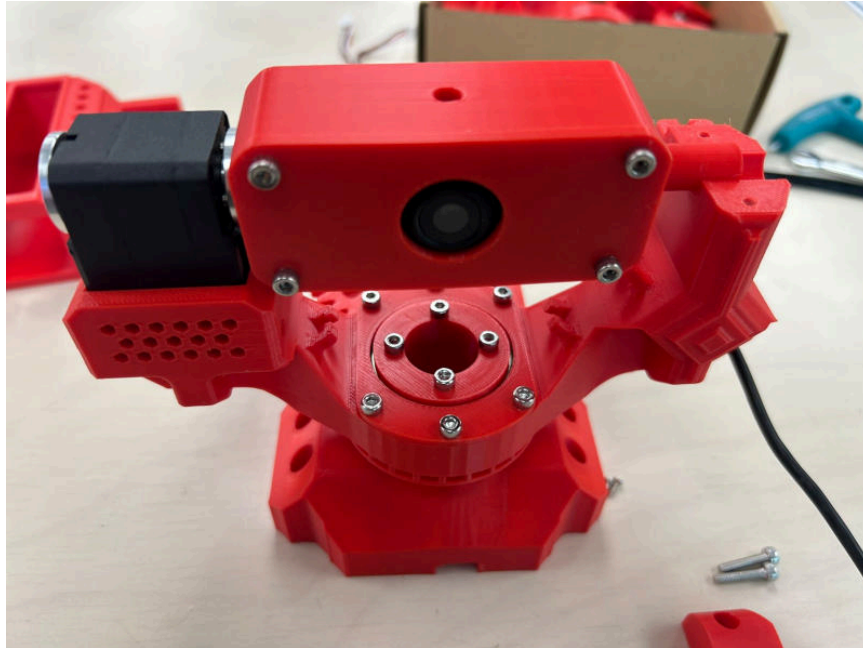
In this part, we would finish building the main part of the tracker

You would need the following materials in this part:



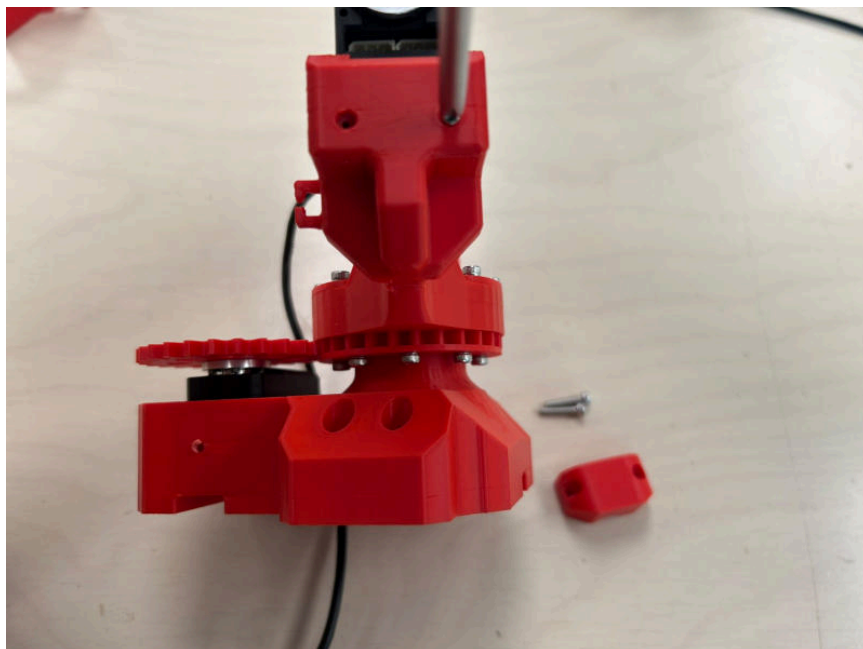
<u>Items</u>	<u>Amount</u>
Finished part from "Part 2"	1
Finished part from "Part 3"	1
3D printed rod mounting part	1
Shape-headed Servo Motor Screws	2
M3 screws (15mm long)	2

Insert the "Finished part from "Part 2"" on the "Finished part from "Part 3"" as shown:



Flip the whole tracker with the servo motor side facing the top

Fix the servo motor's position by screwing it with 2 "Shape-headed Servo Motor Screws" using the screw driver (they are also cross-headed)



After fastening the screws, your part should look like:

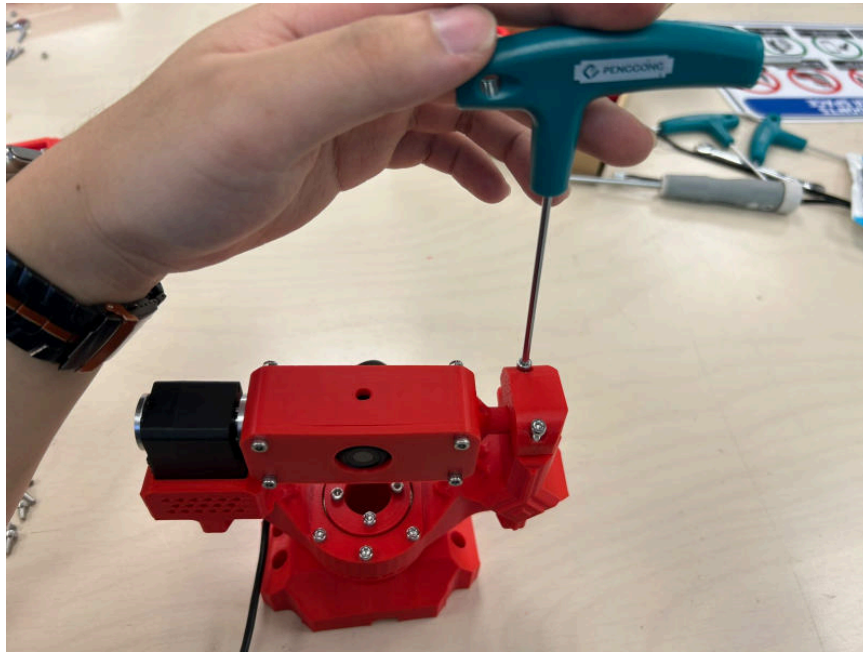
Don't worry if your servo motor does not have the servo motor horn, it does not matter and will not affect our tracker



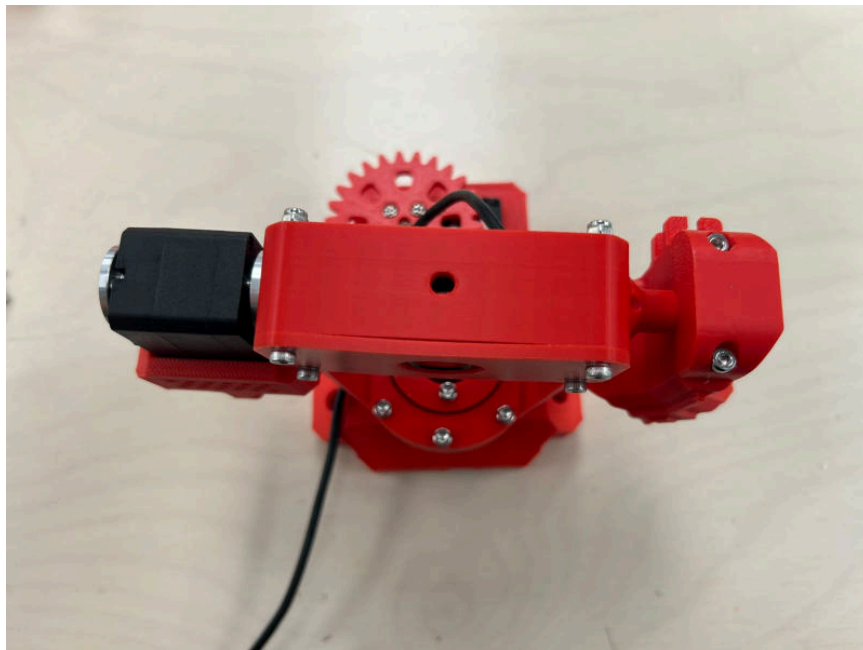
Place the "3D printed rod mounting part" on top of the opposite side of the tracker.

The part should support the rod from the camera housing

Tighten the "3D printed rod mounting part" with 15mm long M3 screws using the hex key



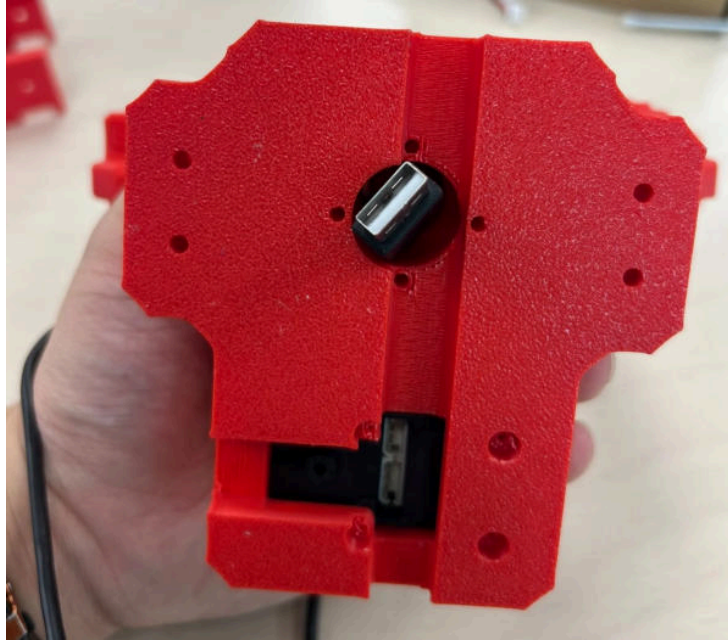
After tightening the screws, you should get a part that looks like this from the top:



Wire the USB camera cable though the center hole of the tracker as shown:



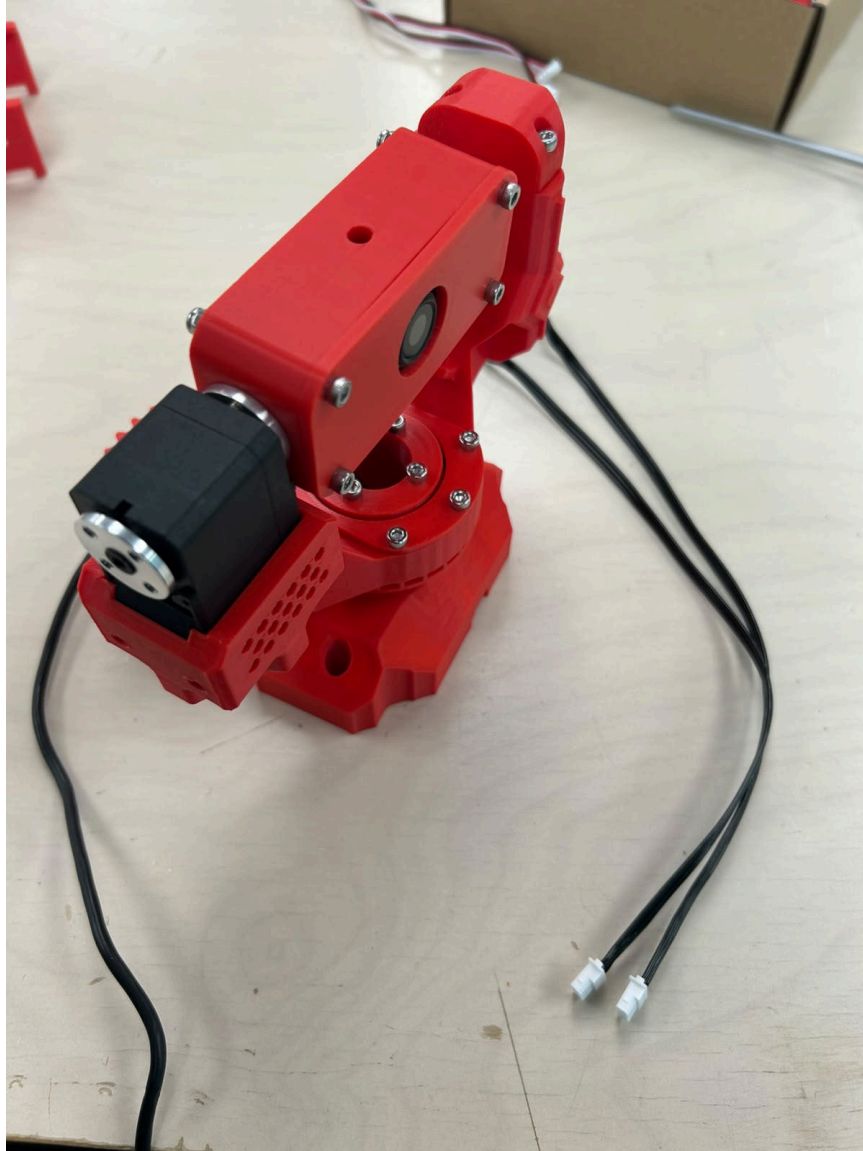
This is how it should look like from the bottom



This is how it should look after you finish all mechanical parts:

You can see now the main tracker part is done with one USB camera cable at the bottom.

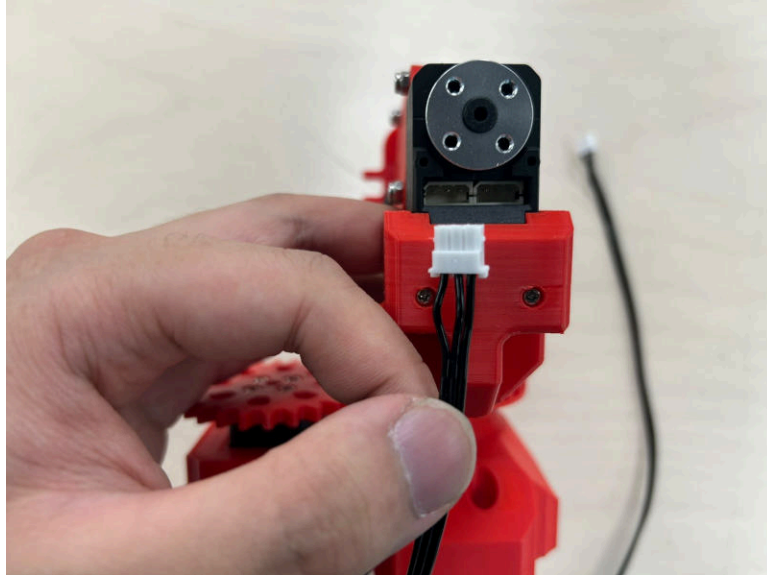
You should also see that there are two motor wires not yet being used yet.



This is why we will start wiring the motors now

Notice the wire direction before plugging in the motor.

The side WITHOUT metal pin holes should face upwards

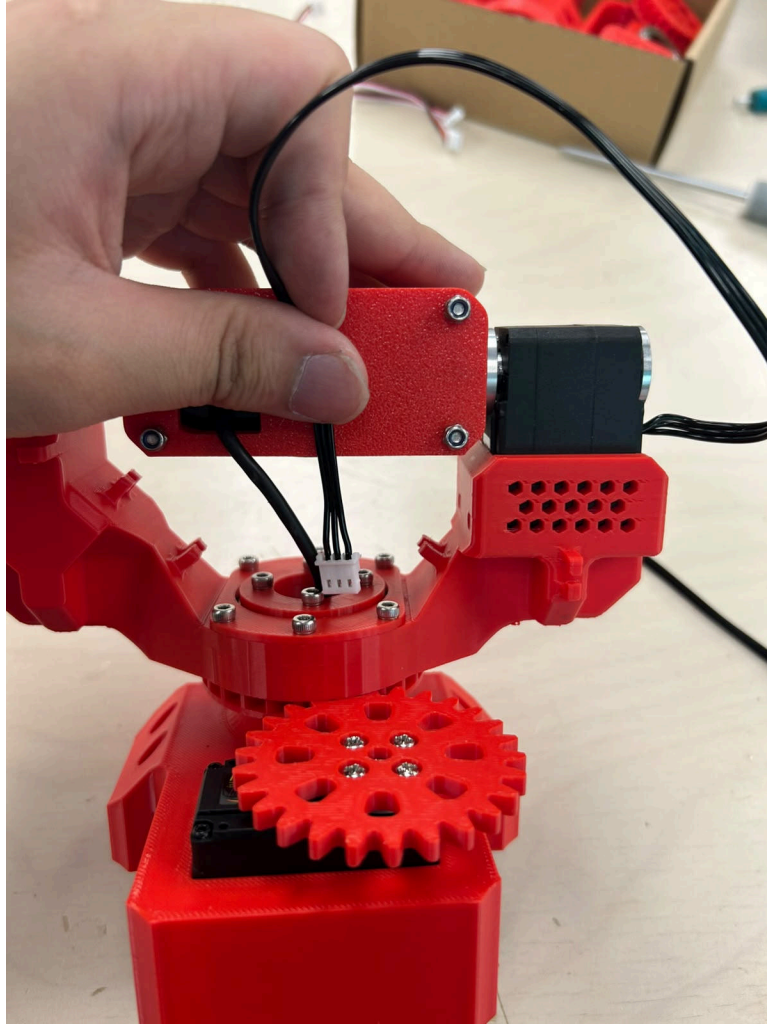


Press the wire into the header on the servo motor

You may have to press it tightly until it plugs in
(usually it requires a "little bit" of force)

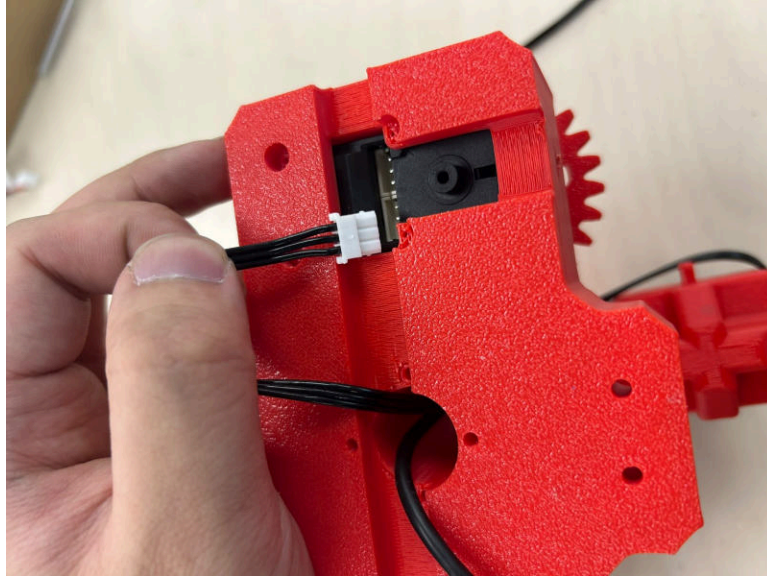


Wire the other end of the servo motor wire through the center tracker hole

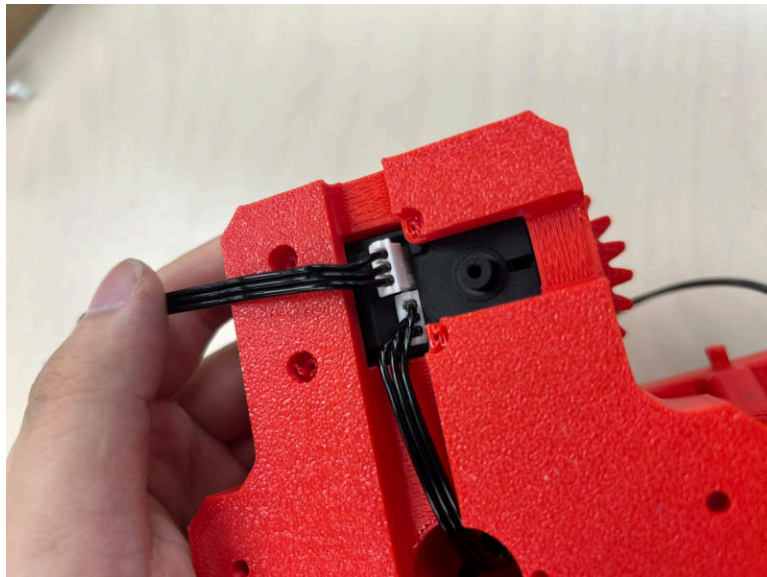


Connect the other side of the servo motor wire to the servo motor on the base

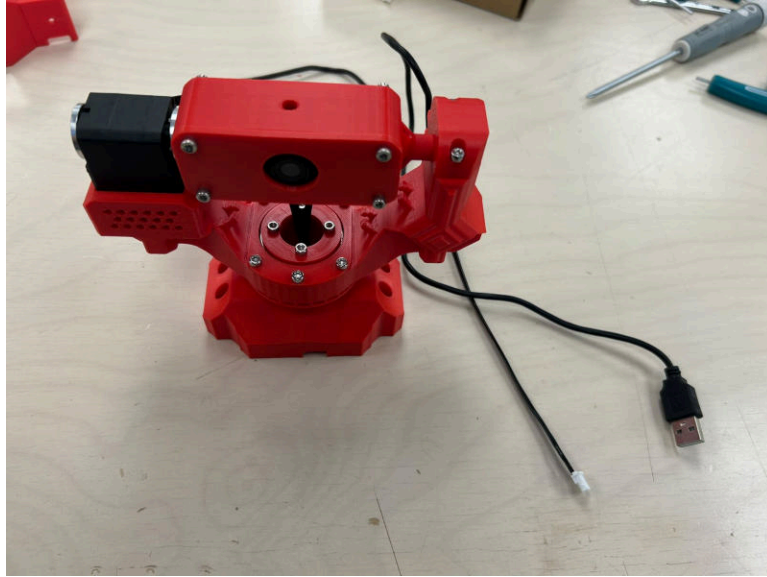
AGAIN!!! remember that the orientation of the wire matters
DO NOT MIX IT UP, please.



Plug in the wire. On the other header of the servo motor, connect one end of the second servo motor wire.



After this, you should get a tracker with two wires/cables extending out
One for the servo motor signals and one for the USB camera signals



Now the main tracker part is done. But we cannot control it yet, we still need the control units

▼ Part 5: Assembling the Control Unit

In this part, we would build and wire the control unit box

Session A: Raspberry Pi

You would need the following materials in this session:

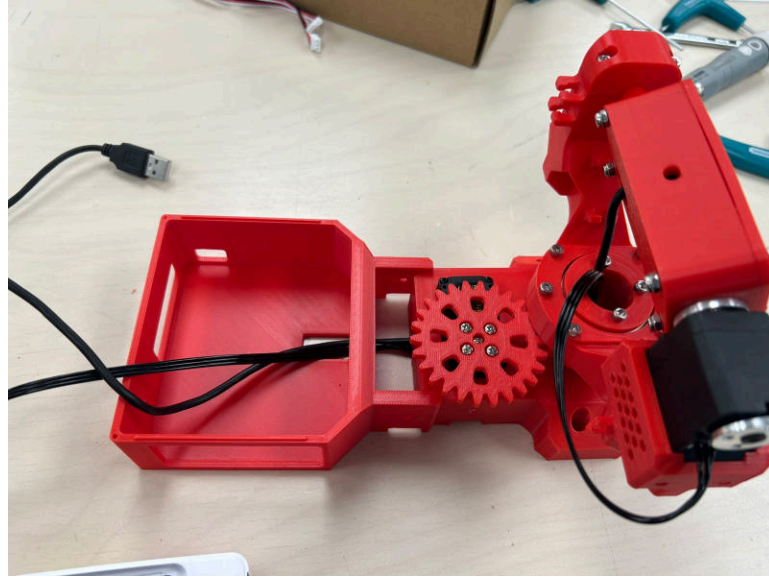


<u>Items</u>	<u>Amount</u>
Finished part from "Part 4"	1
3D printed Electronics Box	1
Raspberry Pi	1

Wire the USB camera wire and Servo Motor wire though the front hole of the "3D printed Electronics Box"



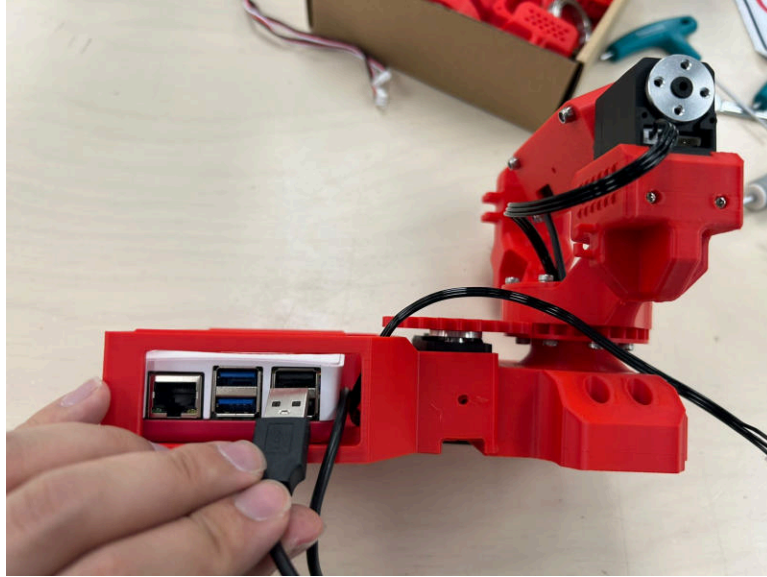
Attach the "3D printed Electronics Box" to the end of the tracker as shown:



Wire the USB camera cable through the side hole and place the Raspberry Pi in the "3D printed Electronics Box"



Plug the USB end of the camera cable to the Raspberry Pi USB slot
You can use any of the four slots, but i choose the right upper one.



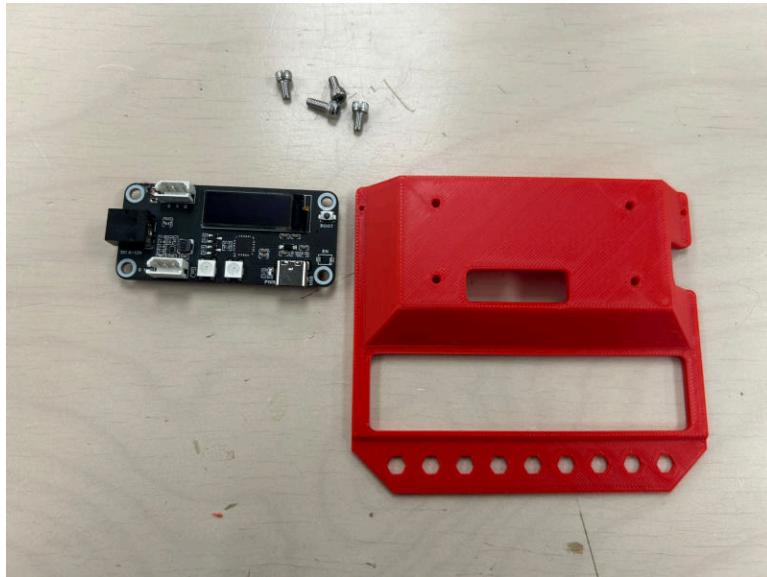
Tidy up the cable and insert it in the "3D printed Electronics Box" for tidiness





Session B: ESP32 servo motor driver

You would need the following materials in this session:

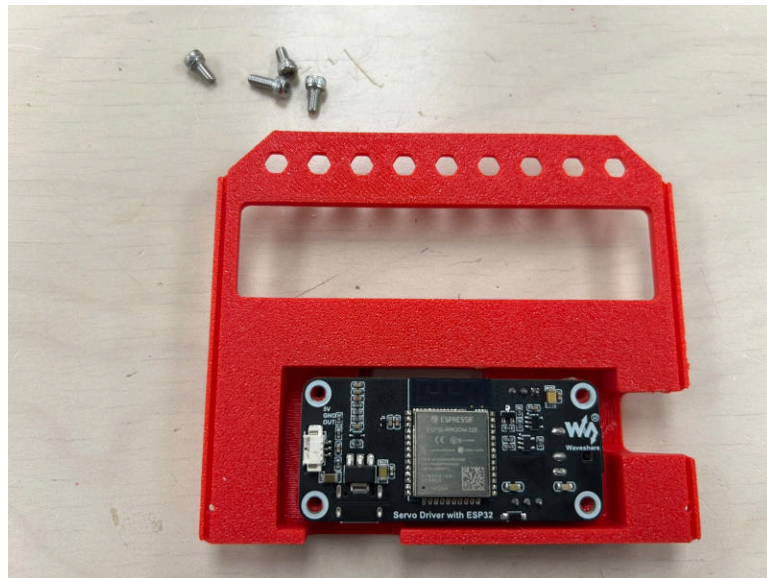


<u>Items</u>	<u>Amount</u>
ESP32 servo motor driver	1
3D printed Electronics Box Cover	1

M3 screws (6/8mm long)	4
------------------------	---

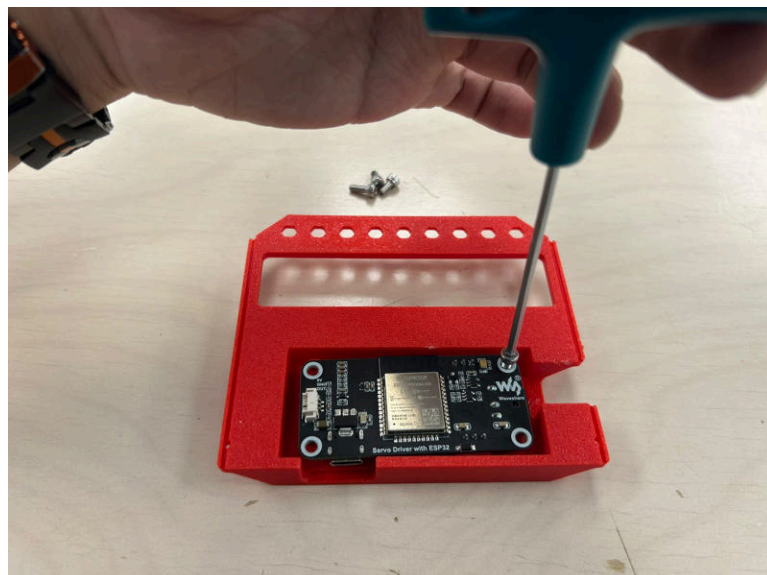
Flip the "3D printed Electronics Box Cover" and place the ESP32 servo motor driver in.

Align the holes in both parts.

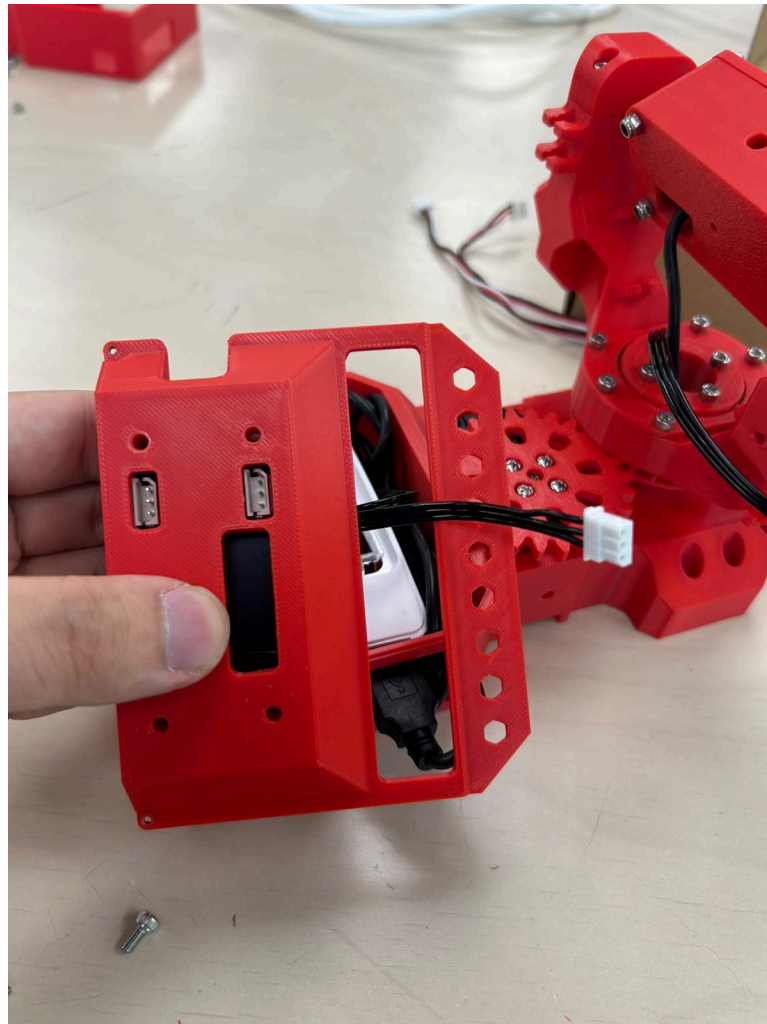


Fix the ESP32 driver in place by screwing in 6/8mm M3 screws with the hex key.

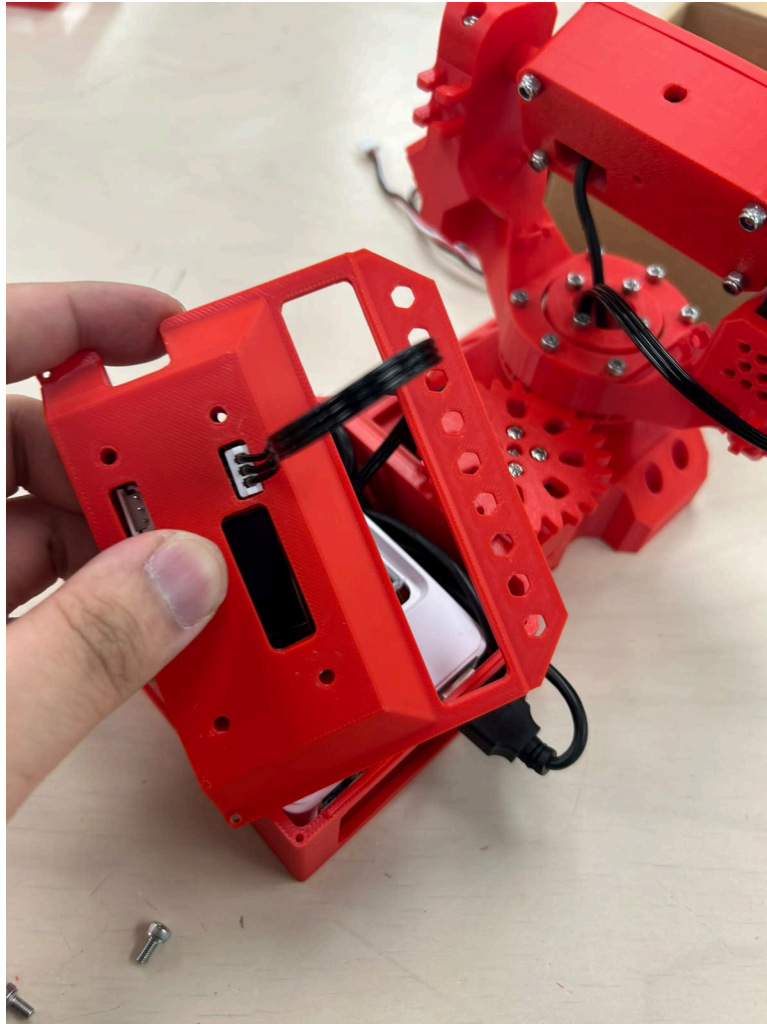
Repeat the same for all four holes.



Pass the servo motor wire through the slot on the "3D printed Electronics Box Cover"

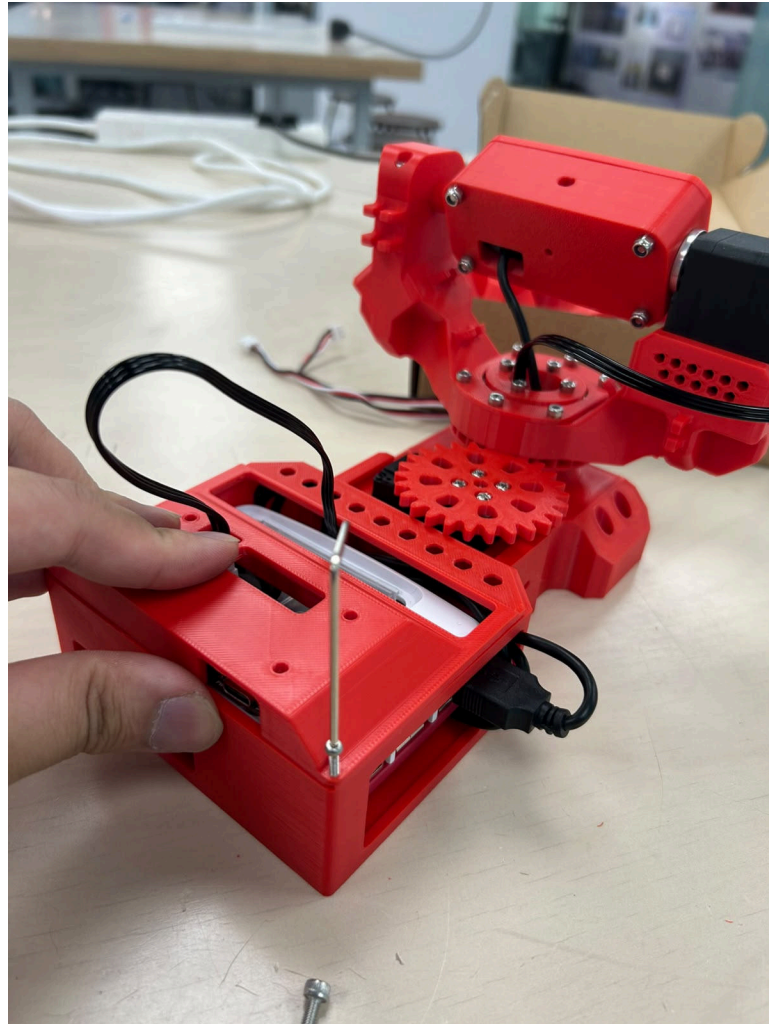


Plug in the servo motor wire to the ESP32 servo motor driver upper port

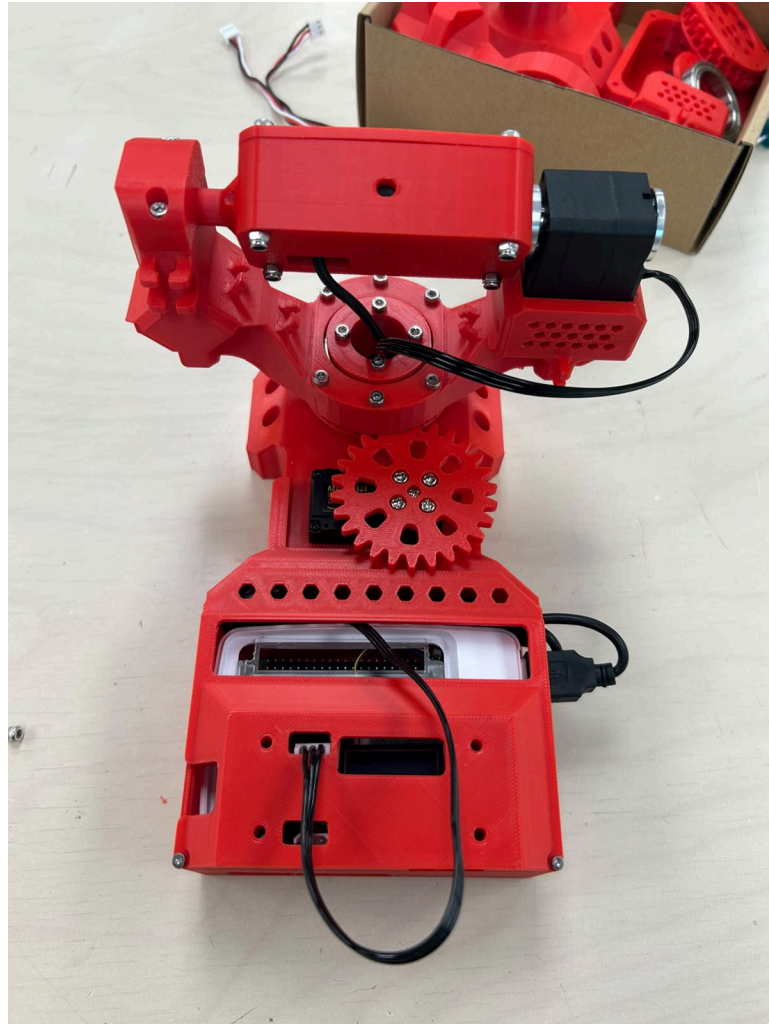


Slide in the cover and fix it using two 8mm long M2 screws.

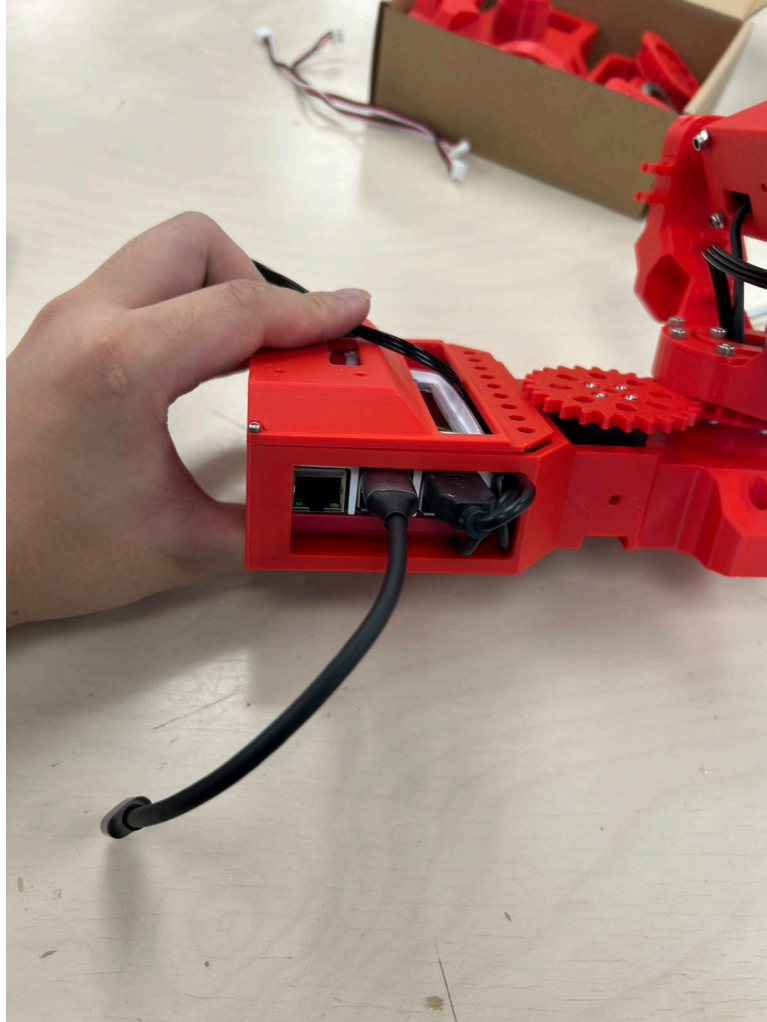
This time you would need a M2 hex key instead of the one with a handle



You tracker should look similar with the you shown below:

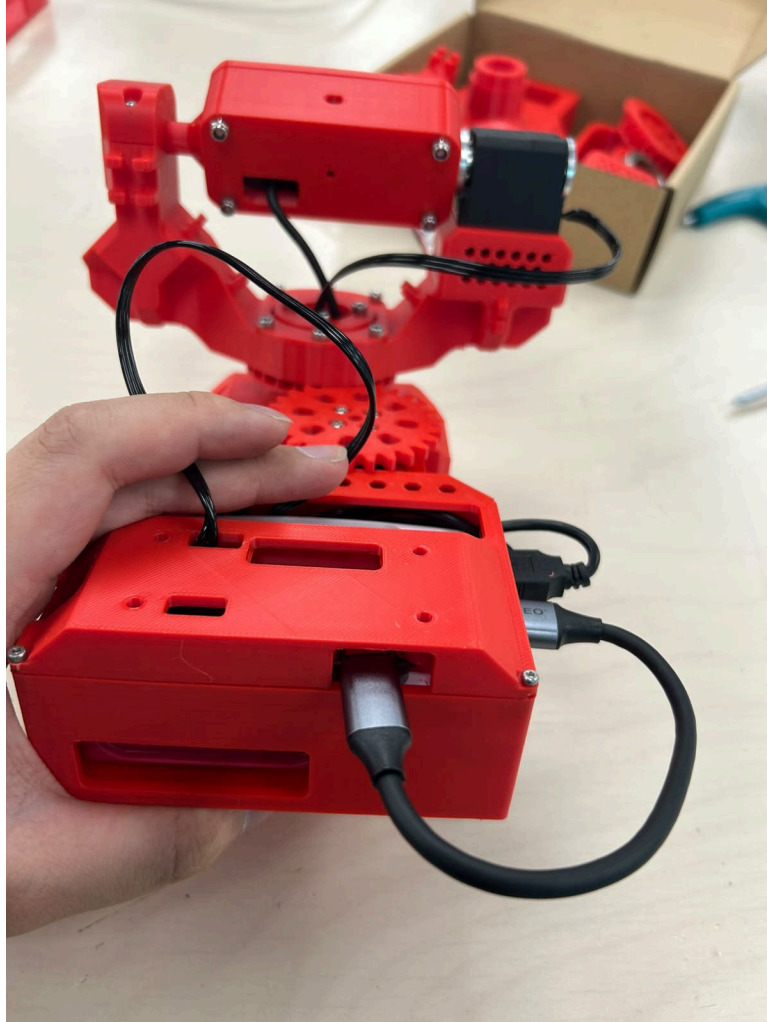


Now, connect the USB end of the USB-C cable to the Raspberry Pi USB slot
(any USB slot among the 4)



On the other end (USB-C) connect to the ESP32 servo motor driver USB-C port.

Be careful not to move it up and down, or you may damage the port



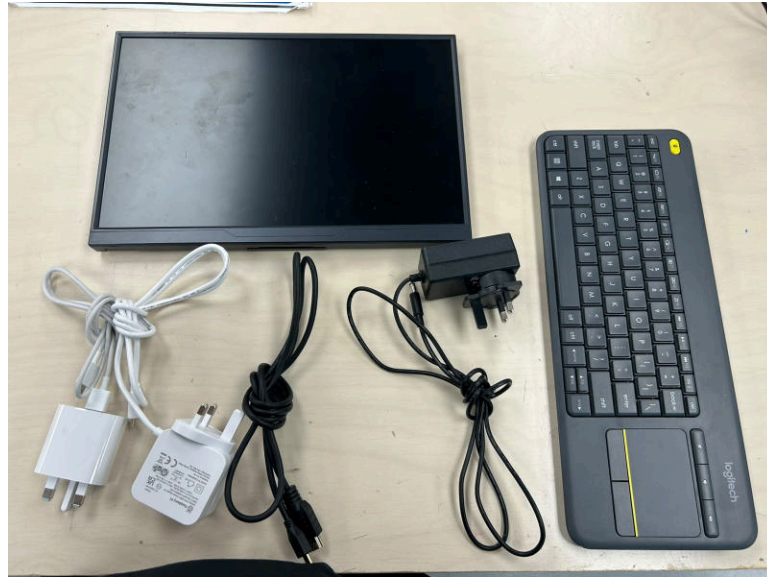
Now you had finished assembling all the parts of the Human tracking robot

Move on to start testing your tracker

Testing Your Human Tracker

▼ Part 1: Wiring the Display & Controls

You would need the following materials in this Part:



<u>Items</u>	<u>Amount</u>
Monitor	1
Keyboard & Mouse	1
Micro-HDMI to HDMI Cable	1
12V Servo Motor Power Supply	1
5V 5A Raspberry Pi Power Supply	1
USB-C power for monitor	1

First, take the Bluetooth adapter for the keyboard & mouse from the back of it. The adapter is located at the cover of the battery slot.

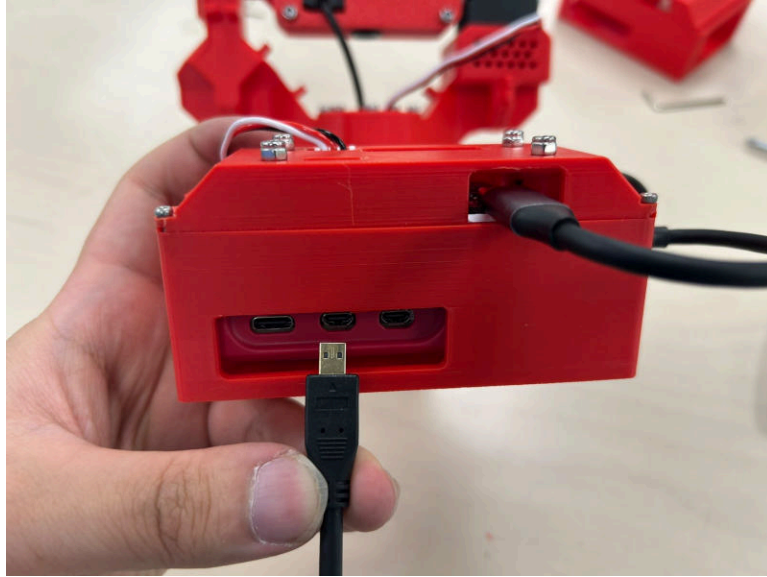


Plug the Bluetooth adapter to the Raspberry Pi USB port

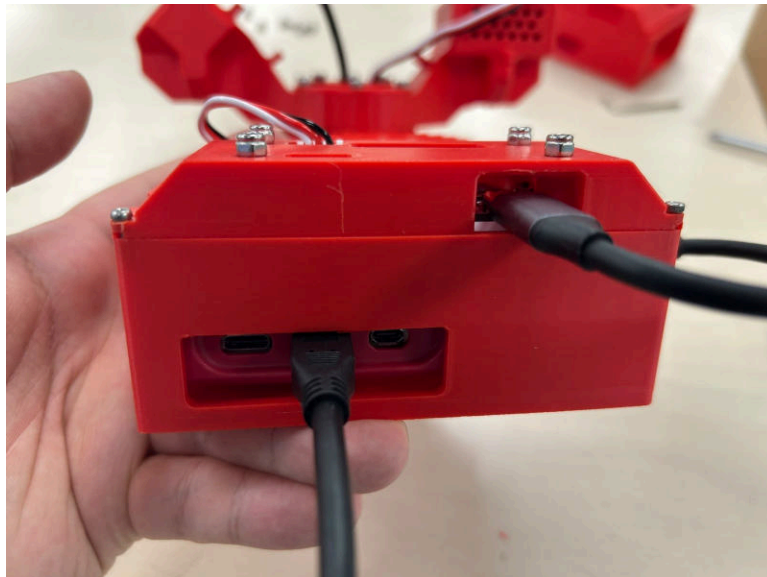


Connect the micro-HDMI end of the HDMI cable to the raspberry HDMI port

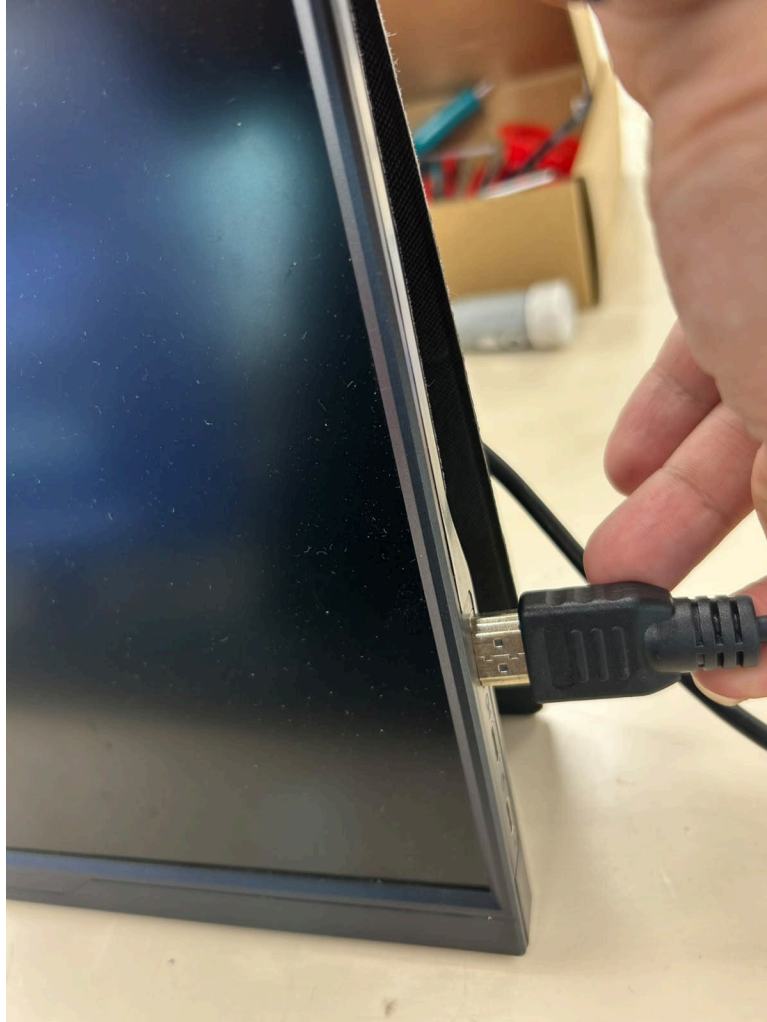
The Raspberry Pi HDMI port is located at the back of the tracker



Plug the HDMI cable in, this is how it should look:

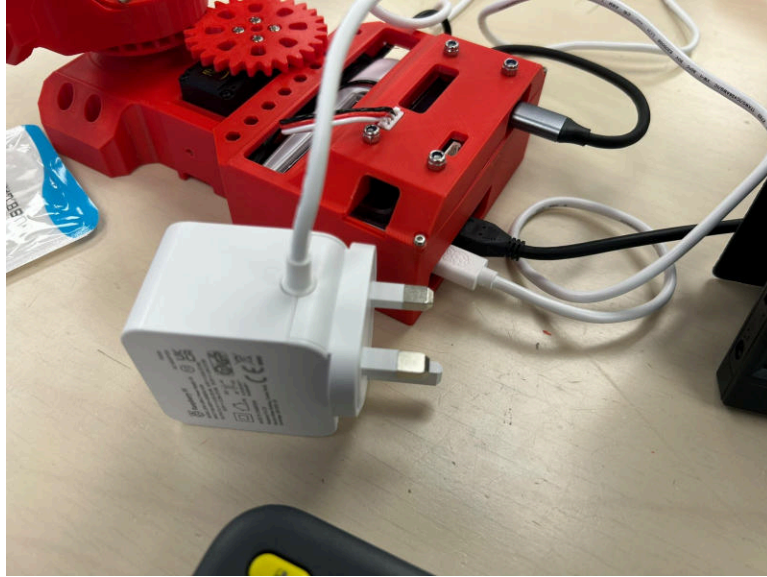


Connect the other HDMI end to HDMI port on the right side of the monitor

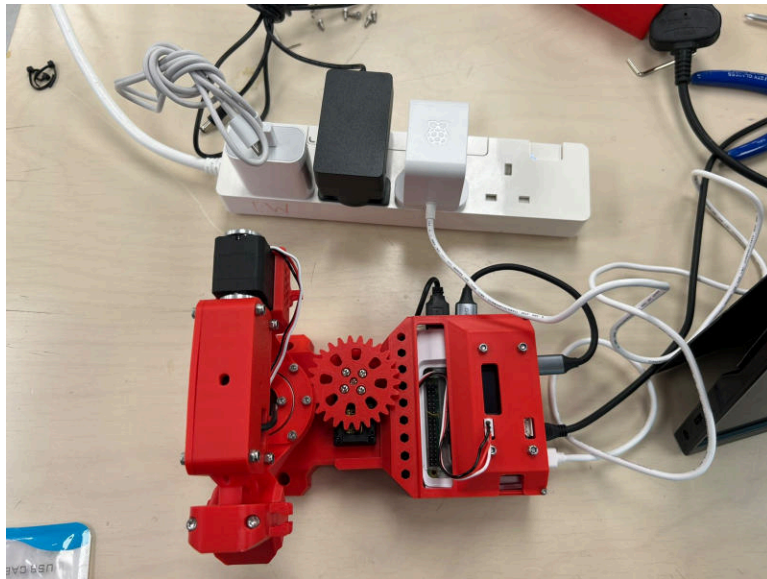


Connect the other USB-C end of the 5V5A Raspberry Pi Power supply to the power port of the Raspberry Pi.

The Raspberry Pi power port is the leftmost port at the back of the tracker



Connect all 3 Power supplies (Servo Motor Power, Raspberry Pi Power & Monitor Power) to a socket.



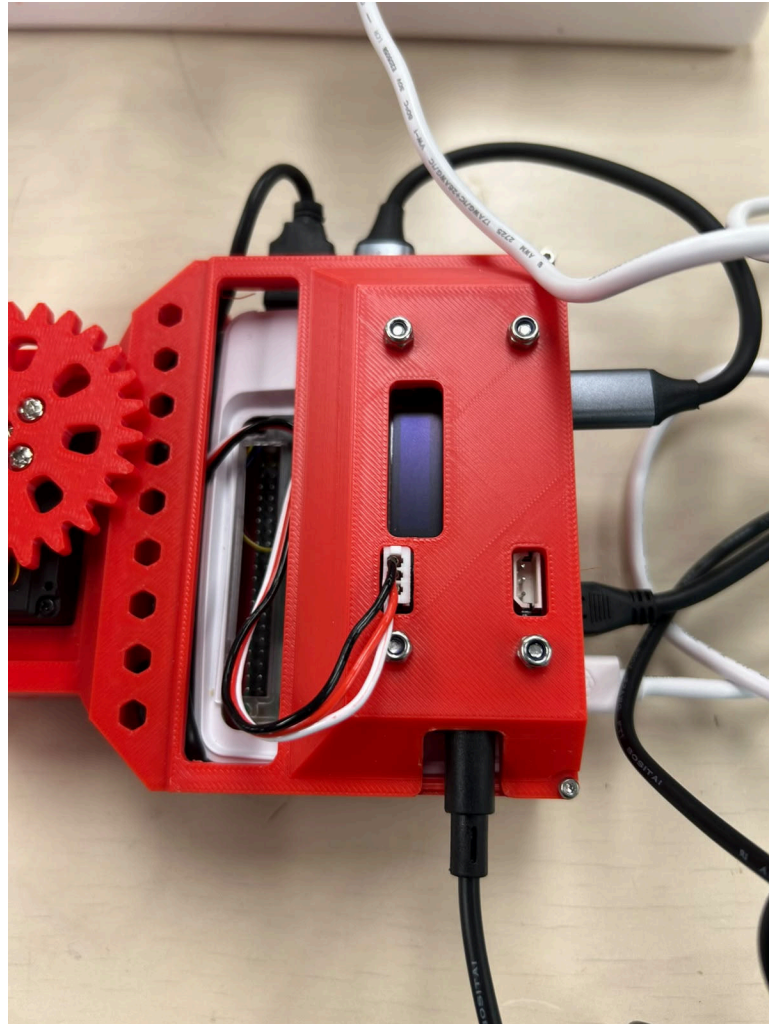
Connect the DC plug of the 12V Servo Motor Power Supply to the ESP32 Servo Motor Driver Power Port

Remember we mounted the servo motor driver board to the cover of the electronics box. The power port is located at the left side of the

box.



Plug it in and it should look like this:



Connect the USB-C Power for the Monitor



You should see that the ESP32 servo motor driver board displays:
"Searching Servos ... MAX_ID 20-Ping:4 Detected: 1 2"

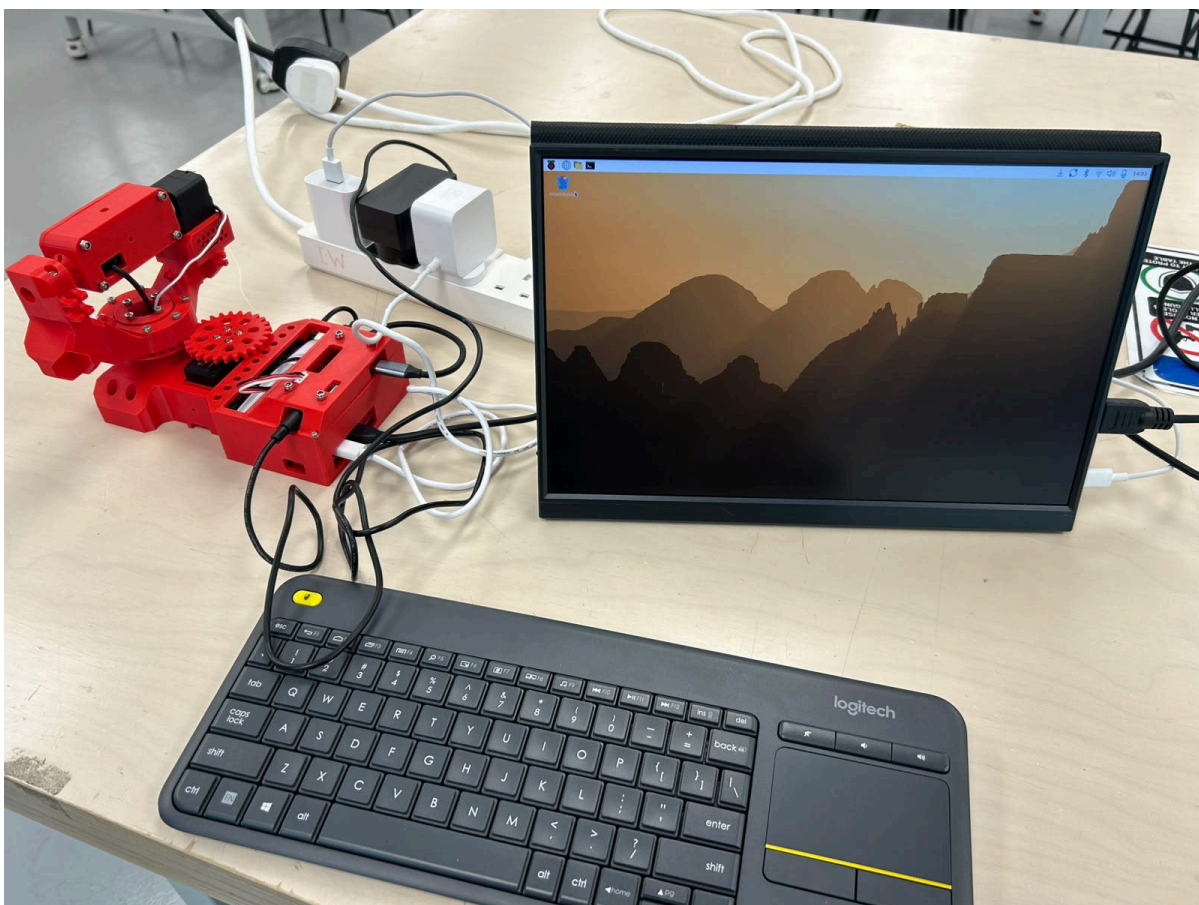
Indicating it found motor ID 1 and Motor ID 2, corresponding to the two motors we have:



It would quickly turn to "SERIAL_FORWARDING" mode automatically, allowing us to control the motors using the Raspberry Pi.



After all of this, you should see the desktop of the Raspberry Pi on the monitor



And now we are finally ready to run code on the tracker robot

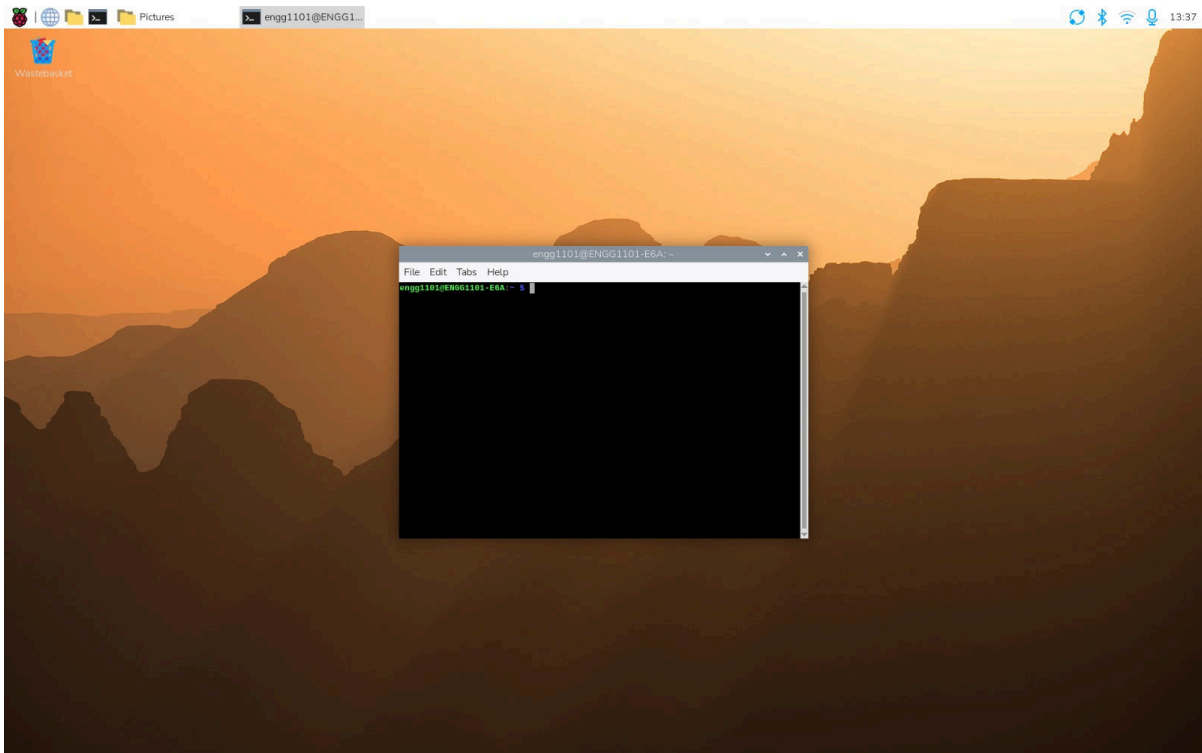
▼ Part 2: Reading the motor values & Basic Controls

Now you should be able to see the desktop of the Raspberry Pi.

Let's try to run some program and control the human tracking robot

First press on `Ctrl + Shift + T` on your keyboard. The terminal will pop up.

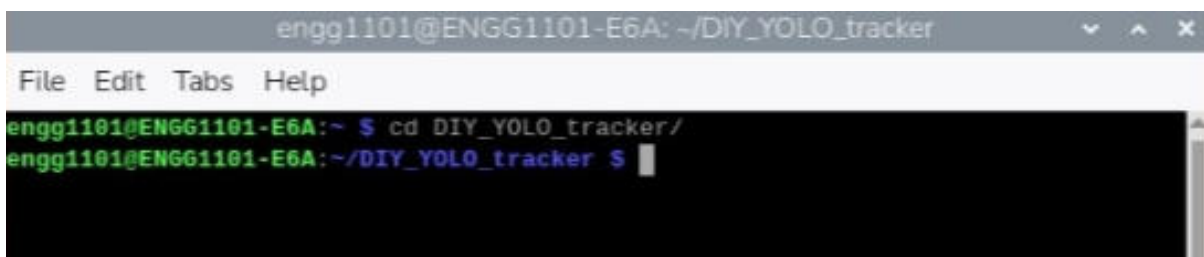
The terminal is the place where you can type in commands to execute programs or computer operations



Click on the terminal (the black window that pops up) and type the command:

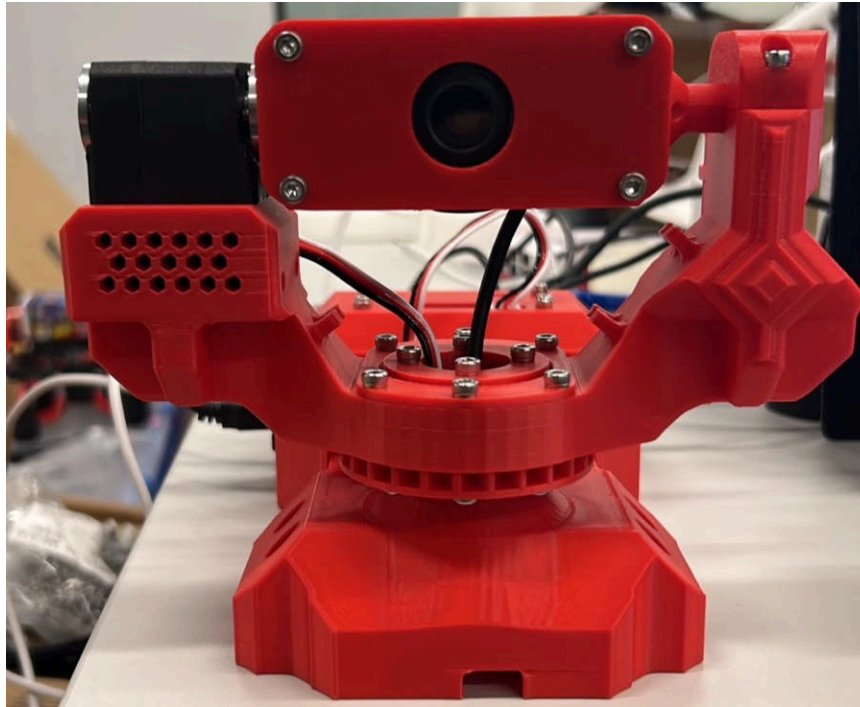
```
cd DIY_YOLO_tracker/
```

Then press the **Enter** key, the terminal should show the following message:



Now, you had just navigate into the folder called "DIY_YOLO_tracker"

Next, center the YOLO tracker robot so that the camera faces the front horizontally, like this:



Type the following command:

```
python3 gimbal_controller.py
```

This command allows you to run the python code

Then press the **Enter** key, the terminal should show the following message:

- Showing the control keys (use left/right arrows to control yaw rotation)
- Current position of the servo motors

```

eng61101@ENG61101-E6A:~ $ cd DIY_YOLO_tracker/
eng61101@ENG61101-E6A:~/DIY_YOLO_tracker $ python3 ENG61101_gimbal_controller.py
Succeeded to open the port
Succeeded to set baudrate

Keyboard control active. Use:
  Left/Right  -> Servo 1  (ID 1)
  Up/Down     -> Servo 2  (ID 2)
  ESC        -> quit

Servo 1: 227 | Servo 2: 2485 █

```

Remember this center value of servos 1&2, we will use it later.

For mine the center position of servo 1 is 227 and center position of servo 2 is 2485

IMPORTANT : RECORD THE INITIAL POSITION OF BOTH MOTORS !!!

IMPORTANT : RECORD THE INITIAL POSITION OF BOTH MOTORS !!!

IMPORTANT : RECORD THE INITIAL POSITION OF BOTH MOTORS !!!

Now you can try pressing the arrow keys on the keyboard to manual control the robot

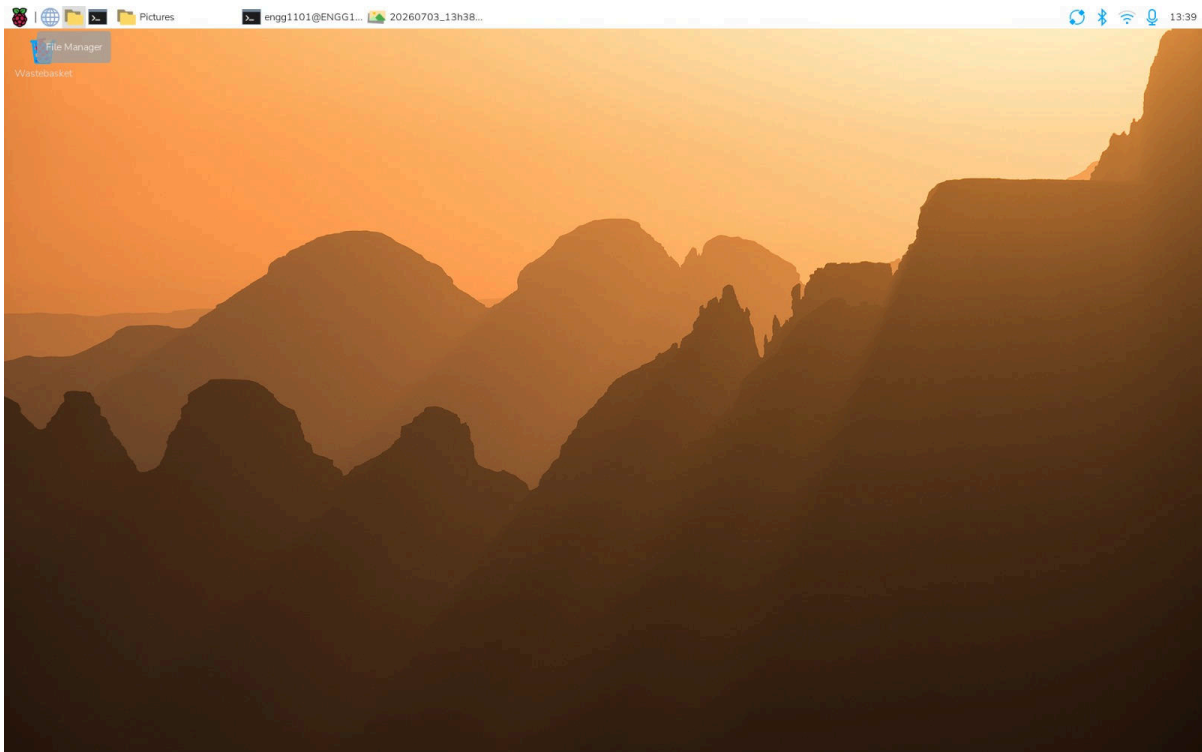
[attachment:7c4da276-b8ed-43e9-b256-b3fb461eb358:WhatsApp_Video_2026-07-03_at_13.56.38.mp4](#)

Controlling the robot manually is not fun, let's try to make it auto

▼ Part 3: Changing the code and running the tracking program

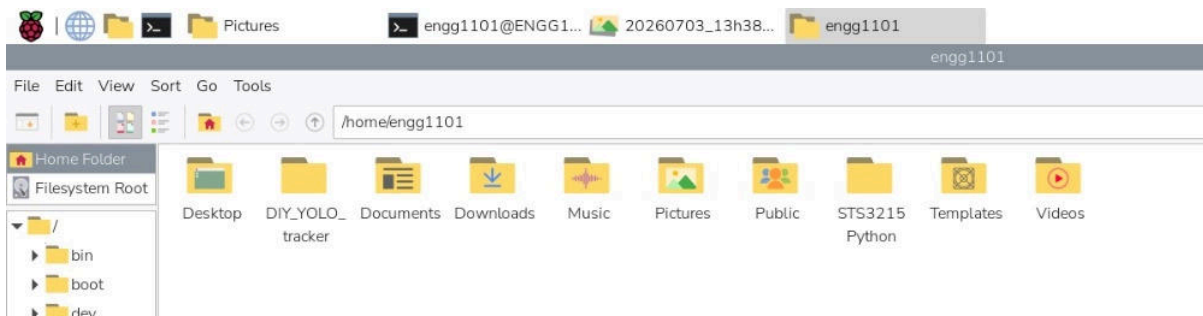
Let's first move back to the desktop.

Then click on the "File Manager" icon on the left upper corner.

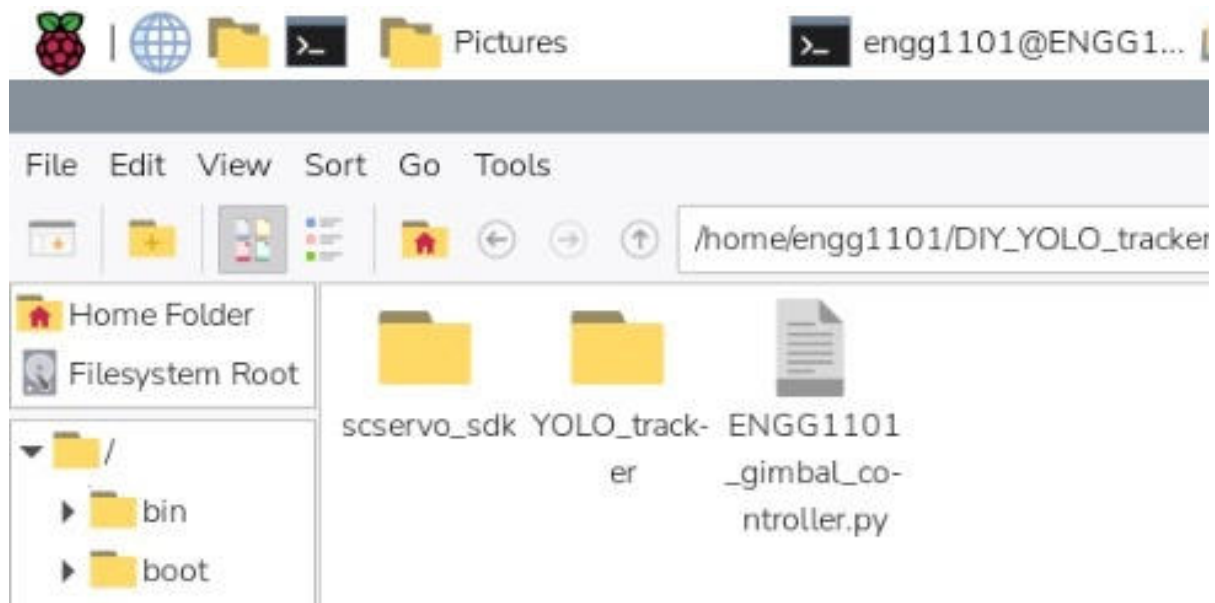


Once the File Manager pops up you should be under the home directory.

Click onto the Folder: **"DIY_YOLO_tracker"**

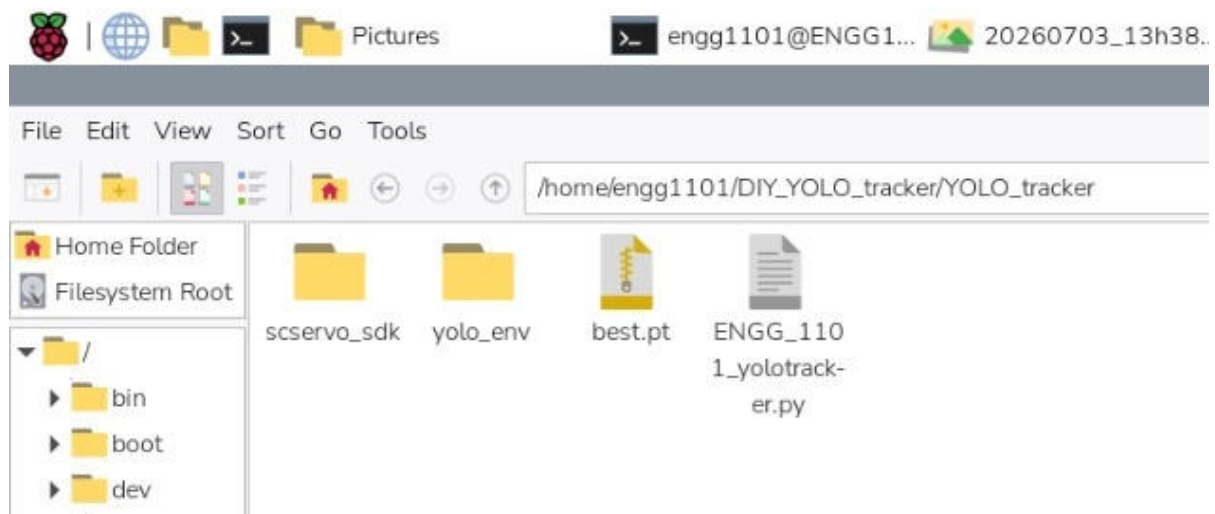


Once you opened the `"DIY_YOLO_tracker"`, you should be able to see the following:



This is where we stored our program in Part 2

Now, click into the `"YOLO_tracker"` folder, you should be able to see:



Click and open the `"yolotracker.py"` python file. The python script should pop up:

```
1 import cv2
2 import torch
3 from ultralytics import YOLO
4 import time
5 import math
6 from collections import deque
7 import numpy as np
8 import sys
9 import os
10
11 sys.path.append(...)
12 from scservo_sdk import *
13
14
15 class SimpleHeadTrackingSystem:
16     def __init__(self, model_path='best.pt', ports='/dev/ttyUSB0', servo_ids=[1, 2],
17                 frame_width=1280, frame_height=720):
18         print(f" * 70")
19         print("PID CONTROLLER HEAD TRACKING - NO OVERSHOOT")
20         print(f" * 70")
21
22         # Load model
23         self.model = YOLO(model_path)
24         self.model.amp = False
25         self.model.fuse()
26
27         print("Model class names: ", self.model.names)
28
29         # Camera settings
30         self.frame_width = frame_width
31         self.frame_height = frame_height
32         self.frame_center_x = self.frame_width // 2
33         self.frame_center_y = self.frame_height // 2
34
35         # Servo limits
36         self.servo_horizontal_min = 0
37         self.servo_horizontal_max = 4095
38         self.servo_horizontal_start = 2036
39         self.current_servo_horizontal = self.servo_horizontal_start
40
41         self.servo_vertical_min = 0
42         self.servo_vertical_max = 4095
43         self.servo_vertical_start = 2500
44         self.current_servo_vertical = self.servo_vertical_start
45
46         # Servo movement settings
47         self.moving_speed = 4095
48         self.moving_acc = 255
49
50         # PID CONTROLLER SETTINGS
```

Now, we need to change the code based on our previous servo center readings
(From Part 2)

Change lines 38 & 43 values to what you recorded in Part 2

(remember for my device is Servo 1 = 227 & Servo 2 =2485)

- Replace the value of `self.servo_horizontal_start` (line 38) to YOUR servo 1 value
- Replace the value of `self.servo_vertical_start` (line 43) to YOUR servo 2 value

```
35 # Servo limits
36 self.servo_horizontal_min = 0
37 self.servo_horizontal_max = 4095
38 self.servo_horizontal_start = 227
39 self.current_servo_horizontal = self.servo_horizontal_start
40
41 self.servo_vertical_min = 0
42 self.servo_vertical_max = 4095
43 self.servo_vertical_start = 2485
44 self.current_servo_vertical = self.servo_vertical_start
45
46 # Servo movement settings
47 self.moving_speed = 4095
```

Click **Ctrl + S** to save the python script.

IMPORTANT : Remember to SAVE

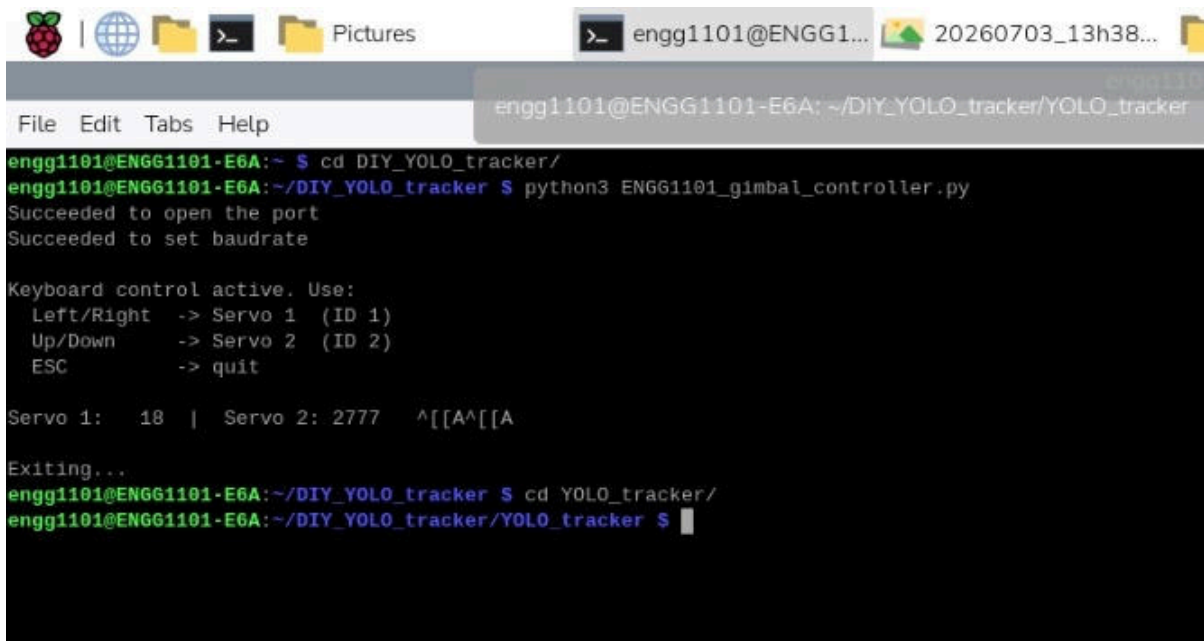
IMPORTANT : Remember to SAVE

IMPORTANT : Remember to SAVE

Now, head back to the terminal by clicking the terminal icon on the top bar.

Type the following command:

```
cd YOLO_tracker
```



```
enggg1101@ENGG1101-E6A:~ $ cd DIY_YOLO_tracker/
enggg1101@ENGG1101-E6A:~/DIY_YOLO_tracker $ python3 ENGG1101_gimbal_controller.py
Succeeded to open the port
Succeeded to set baudrate

Keyboard control active. Use:
  Left/Right  -> Servo 1  (ID 1)
  Up/Down     -> Servo 2  (ID 2)
  ESC        -> quit

Servo 1:  18 | Servo 2: 2777  ^[[A^[[A

Exiting...
enggg1101@ENGG1101-E6A:~/DIY_YOLO_tracker $ cd YOLO_tracker/
enggg1101@ENGG1101-E6A:~/DIY_YOLO_tracker/YOLO_tracker $
```

We are now inside the `YOLO_tracker` directory where we just modified the code.

Type the following command:

```
source yolo_env/bin/activate
```



```
(yolo_env) enggg1101@ENGG1101-E6A:~/DIY_YOLO_tracker/YOLO_tracker $ source yolo_env/bin/activate
(yolo_env) enggg1101@ENGG1101-E6A:~/DIY_YOLO_tracker/YOLO_tracker $ python3 ENGG_1101_yolotracker.py
Using serial port: /dev/ttyUSB0
Target resolution: 1280x720
=====
PID CONTROLLER HEAD TRACKING - NO OVERSHOOT
=====
```

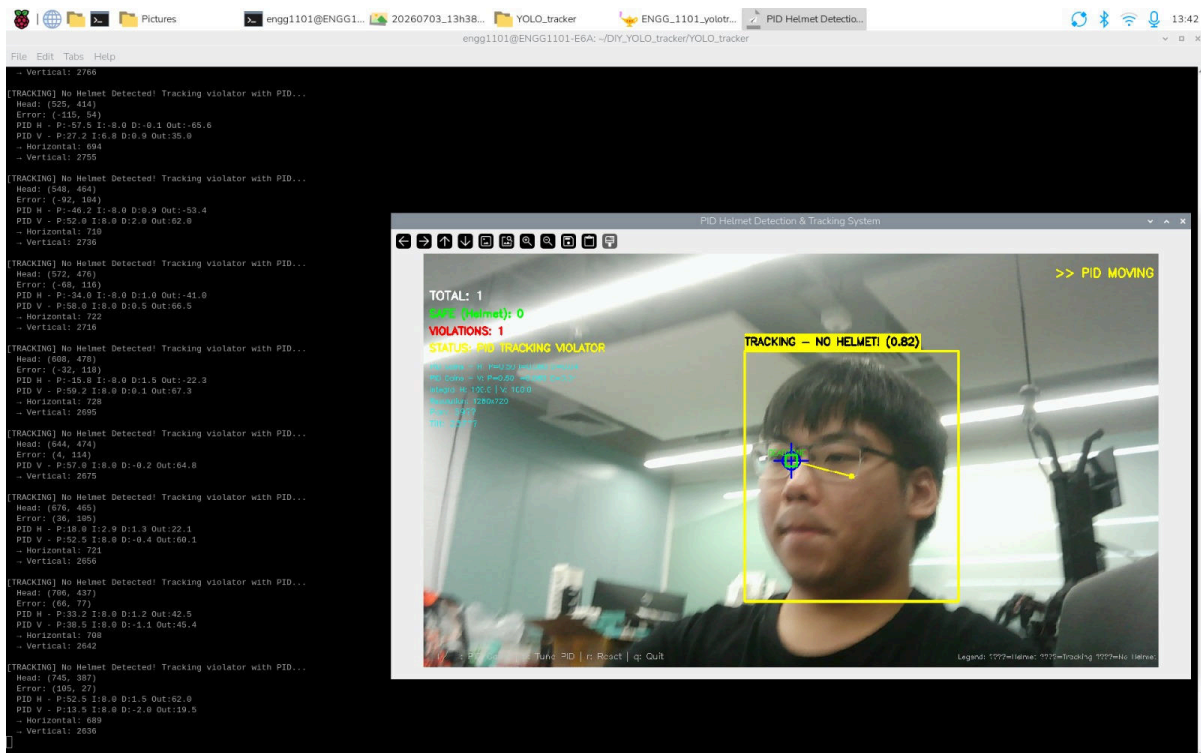
You should see `(yolo_env)` in the front of the line, stating we activated the virtual environment which includes the libraries we need to use.

Type the following command:

```
python3 yolotracker.py
```

This runs the tracker program and enjoy playing with the robot

You should be able to see the camera view popping up and a cross mark appears to track your head (only heads, not the whole body)



Yea, and that is me.

Great shoutout and thanks to Dr. Ryan Wang for setting up the YOLO tracker topic for ENGG1101 mini-projects which lay the foundations of this workshop

Also, a lot of thanks to 2026 Semester ENGG1101 Group M5 for providing the base code and fundamental concepts during their mini-project developments.

Summary

What you have done in this workshop:

1. Assemble a YOLO Human Tracker Robot successfully as a team.
2. Modify Python Code according to servo initial positions
3. Enjoy playing with the tacker robot
4. Start your journey as an engineer, Welcome to the family.

